

TODO

- ☐ proof read, stavefeil, flow, struktur, ordlegging akademisk tone. formler, gjenta til fornøyd
- ☐ figurer
- ☐ referanser
- ☐ metode og resultater handcrafted/kopierte vekter
- ☐ metode og resultater neuromorfisk læring
- ☐ discusion
- ☐ conclusion
- ☐ abstract
- ☐ siste proof read

Wordcount: 17687

NEUROMORPHIC COMPUTING

With Spiking Neural Networks

Brage Wiseth

Departement of Informatics
Faculty of Mathematics and
Natural Sciences

Philip Haflinger - Supervisor
Yngve Hafting - Co-Supervisor



ABSTRACT

The development of modern Deep Learning has achieved unprecedented performance across various domains, yet it remains fundamentally bottlenecked by the energy and memory inefficiencies of the von Neumann architecture. To address these limitations, this thesis investigates Neuromorphic Computing with Spiking Neural Networks (SNNs) as a biologically plausible, highly energy-efficient alternative. By shifting from synchronous, continuous-value matrix multiplications to asynchronous, event-driven sparse computations, neuromorphic systems emulate the physical principles of the biological brain.

This work explores the implementation of these principles on standard CPU/GPU hardware. Two primary methodologies are developed and evaluated on visual classification tasks: (1) an inference-optimized SNN that translates weights from a conventionally trained Fully Connected Network (FCN) using Time-To-First-Spike (TTFS) temporal encoding, and (2) an unsupervised, biologically inspired learning simulator incorporating structural plasticity (dynamic synaptogenesis and pruning). The results demonstrate the viability of temporal coding and local learning rules in extracting meaningful features from visual stimuli, highlighting the potential of neuromorphic algorithms to drastically reduce the computational footprint of artificial intelligence.

ACKNOWLEDGEMENTS & DECLARATIONS

I would like to thank my supervisor and the very kind and helpful community at ROBIN and NANO laboratories at the department of informatics. I would like to thank professor Farad at SDU

DECLARATION OF THE USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service GPT UiO, developed by UiO IT department, has been used to improve the content of the report/assignment. A first version of the work was pasted in its entirety, and the model was given the prompt [rewrite this text to make the language more lively.] The text was then iterated a few times through the model where new prompts were used to get the correct structure of the text. The final result was cut out, fact-checked, and partly rewritten by the author(s).

CONTENTS

1 • Introduction	5
2 • History & Developments	7
2.1 • The Perceptron	8
2.2 • Deep Learning	9
2.3 • Birth Of Neuromorphic	12
3 • Biological Principles	14
3.1 • Neuron Structure & Function	14
3.2 • Action Potential & Spike Trains	16
3.3 • Neuron Models	18
3.4 • Neural Coding	22
3.5 • Neural Networks	26
3.6 • Biological Learning	29
4 • Technical Details Of Machine Learning	33
4.1 • Optimization	33
4.2 • Deep Learning Framework	35
4.3 • Why Is Deep Learning Inefficient?	37
4.4 • Principles of Neuromorphic Engineering	39
4.5 • Training Spiking Networks: The Non-Differentiability Problem	40
4.6 • Neuromorphic Hardware Techniques	41
4.7 • State of the Art: Sensors and Simulation	42
5 • Method	44
5.1 • Dataset & Pre-processing	44
5.2 • Network Architecture	45
5.3 • Information Representation	46
5.3.1 • Encoding	48
5.3.2 • Decoding	49
5.4 • SNN Implementation Details	52
5.5 • Training Methodologies	52
5.5.1 • Weight Transfer	53
5.5.2 • STDP Inspired	53
5.6 • Simulation Setup	54
5.7 • Evaluation Metrics	55
6 • Results	56
6.1 • Inference	56
6.2 • Training	59
7 • Discussion	62
7.0.1 • Perceptron equivalence	62
7.1 • Future Work	66
8 • Conclusion	67
Bibliography	69

GLOSSARY

AI – Artificial Intelligence 5, 6, 6

ANN – Artificial Neural Network 10, 27, 40, 45, 46

action potential: Voltage potential in the neuron cell membrane that propagates through the neurons axon and to its synapses 15

CNN – Convolutional Neural Network 10, 27, 37, 45, 45

CPU – Central Processing Unit 6

DAG – Directional Acyclic Graph 27

DL – Deep Learning 33, 37

FCN – Fully Connected Network 45, 45

GLIF – Generalized Leaky Integrate and Fire 21, 21

GPT – Generative Pre-trained Transformer 11

GPU – Graphics Processing Unit 5, 6, 10

LIF – Leaky Integrate and Fire 19, 20, 20, 21

LLM – Large Language Model 11

LSTM – Long Short-Term Memory 11

LTD – Long-Term Depression 31, 31

LTP – Long-Term Potentiation 31, 31, 32

MLP – Multi Layer Perceptron 9, 9, 10, 45

PSC – Post Synaptic Current 26

RNN – Recurrent Neural Network 11

SNN – Spiking Neural Network 26, 40, 45, 46

STDP – Spike-Timing-Dependent Plasticity 31, 31, 31, 31, 40, 44, 45, 53

TTFS – Time-To-First-Spike 23, 24, 24, 24, 46, 47, 48, 51, 62, 62, 62, 63

VLSI – Very Large Scale Integration 12

WTA – Winner-Take-All 24, 28, 29, 46

The number of words in this block is 10409 and there are 63891 letters.

1 • INTRODUCTION

The development of intelligent machines is a significant objective in modern science and engineering. While the concept has historical roots in philosophy and early automata, the field has transitioned from speculative theory to practical application. Currently, artificial intelligence is central to technological and economic development. Understanding the mechanisms of intelligence and reproducing them in synthetic systems offers the potential for improved analysis of biological minds and the creation of tools for applications ranging from personalized medicine to automated scientific discovery.

In recent years, great strides have been made towards this goal. Deep Learning, which utilizes multilayered neural networks, has exceeded previous performance benchmarks. These systems have demonstrated high proficiency in tasks previously limited to human capability. For example, AlphaFold has addressed complex problems in protein folding [1], reinforcement learning agents have mastered the complexity of games such as Go [1], and Large Language Models have shown capabilities in text generation that approach human fluency. Consequently, Artificial Intelligence (AI) is increasingly viewed as a general-purpose technology that may influence societal infrastructure.

However, despite these advances, there are significant limitations to the current approach. The success of modern deep learning relies heavily on scaling, which involves increasing data volume and computational power. This strategy is approaching physical and economic boundaries. Training state-of-the-art models consumes substantial energy and results in a large carbon footprint [1]. Although specialized hardware allows for more efficient computations, the underlying architecture and algorithms impose an intrinsic limit on scalability independent of the underlying hardware. Furthermore, the requirement for massive datasets presents challenges in sourcing and curation. Additionally, evidence suggests that this scaling approach yields diminishing returns. Models often function as statistical correlation engines; they lack common-sense reasoning, struggle with out-of-distribution generalization, and are prone to brittle failure modes [1].

These limitations are evident when comparing artificial systems to biological intelligence. The human brain demonstrates that high-level intelligence is possible without massive energy consumption or dataset sizes. The brain operates on approximately 20 watts [1]. With this limited energy budget, it manages biological functions, processes real-time multi-sensory data, and performs abstract reasoning. In contrast, deep learning models require Graphics Processing Unit (GPU) clusters with significantly higher power requirements to match a fraction of these capabilities. There is also a discrepancy in learning efficiency. Deep learning models are

sample-inefficient, often requiring vast numbers of examples to learn a representation. Biological systems, however, are capable of “one-shot” or “few-shot” learning and can acquire new information without catastrophic forgetting. This suggests the inefficiency of current AI is a paradigmatic issue rather than just an engineering problem.

The proposed direction for addressing these issues involves biological inspiration in both hardware and algorithm design, specifically Neuromorphic Computing. This field attempts to engineer computer architectures that mimic the biological structure of the nervous system. Unlike traditional AI, which runs as software on general-purpose hardware, neuromorphic engineering aims to align the algorithm with the physical substrate. It moves away from clock-driven processing toward asynchronous, event-driven systems. In this paradigm, information is encoded as sparse, discrete events or “spikes.” Similar to biological neurons, a neuromorphic processor consumes minimal energy when inactive, processing information only when triggered. This approach is being pursued by both industrial groups, such as Intel’s Loihi [1], and academic projects like SpiNNaker [1]. These systems represent a shift from calculation-based machines to those capable of real-time adaptation.

Although neuromorphic systems achieve optimal performance on co-designed platforms—where the algorithm is embedded directly into the hardware—there is significant value in executing neuromorphic algorithms on traditional von Neumann architectures. In this thesis, we explore biologically inspired algorithms deployed on traditional Central Processing Unit (CPU) and GPU hardware. We examine how event-driven, biologically plausible computation can address limitations in scalability, data efficiency, and energy consumption, even when simulated on standard processors. We present approaches for efficient information coding and learning algorithms inspired by neural mechanisms. Concretely, this thesis aims to:

- **Investigate Sparse Information Flow:** Explore how information can be encoded and processed using sparse, asynchronous events (spikes) within a neural network.
- **Develop Biologically Inspired Learning:** Explore and evaluate learning algorithms that are suitable for such networks, adhering to the constraints of locality and efficiency.

The succeeding chapter lays the historical and theoretical foundation, covering early neuroscience and the development of artificial neural networks based on simple models of the brain. Following this, we review relevant modern neuroscience literature, extracting key concepts that will inform the methodology. We also provide a concise overview on machine learning concepts and frameworks. Finally, we detail the implementation of these principles in a neuromorphic context and evaluate their performance against standard benchmarks.

2 • HISTORY & DEVELOPMENTS

Historically, the understanding of neural tissue was dominated by the reticular theory, which claimed that the brain consisted of a continuous, fused network of nerve fibers. This paradigm was fundamentally challenged by the work of Santiago Ramón y Cajal. Through the application of novel staining techniques, Cajal established the neuron doctrine, demonstrating that the nervous system is composed of discrete, individual cells [1]. Building on these findings, Heinrich Wilhelm Gottfried Von Waldeyer-Hartz proposed the “Neuron Doctrine” and coined the term “neurons” to describe these discrete cells. Subsequent analysis using electron microscopy has provided irrefutable validation of this discrete cellular structure.

The conceptualization of the brain as a collection of discrete units facilitated the development of mathematical models describing neural function. In 1943, Warren McCulloch and Walter Pitts published *A Logical Calculus of the Ideas Immanent in Nervous Activity*, introducing the first formal model of the neuron.

The McCulloch-Pitts (M-P) neuron abstracted biological complexity into a binary decision device governed by the following logic:

- **Inputs:** The neuron receives multiple binary inputs, weighted as either excitatory or inhibitory.
- **Summation:** The unit calculates the linear sum of these weighted inputs.
- **Thresholding:** If the aggregate sum exceeds a fixed threshold, the neuron outputs a 1 (firing); otherwise, it outputs a 0 (silence).

McCulloch and Pitts demonstrated that networks of these units could theoretically compute any logical operation (AND, OR, NOT) [1]. This abstraction established the foundational link between biological processes and digital logic, suggesting that neural function could be replicated in electronic hardware. Consequently, the M-P neuron serves as the common ancestor for both computational neuroscience and artificial intelligence.

However, despite its theoretical utility, the original M-P model presented significant functional limitations. The connectivity was static, requiring circuits to be manually designed rather than learned. Furthermore, the restriction to binary weights precluded the modeling of graded signal intensity, preventing the system from capturing the nuance of real-world sensory input.

In 1949, Donald Hebb addressed the critical issue of plasticity in his work *The Organization of Behavior*. He proposed a theoretical mechanism for synaptic modification, now known as Hebbian learning, which provided a biological basis for how neural networks could adapt over time. Hebb postulated:

“Let us assume that the persistence or repetition of a reverberatory activity (or “trace”) tends to induce lasting cellular changes that add to its stability. ... When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency, as one of the cells firing B, is increased” [1].

This principle is colloquially summarized as “neurons that fire together, wire together” [1]. Crucially, this describes a local and decentralized learning rule; a synapse does not require a global error signal or external supervision to adjust. It requires only the correlation between the pre-synaptic input and the post-synaptic output. The convergence of the M-P architectural model and the Hebbian plasticity framework established the prerequisite conditions for the development of modern neural networks.

2.1 • THE PERCEPTRON

In 1957, Frank Rosenblatt advanced these theoretical concepts by engineering the Perceptron. The “Mark I Perceptron” was a hardware implementation of the neural model, distinguished by a crucial innovation: a weight-adjustment mechanism based on Hebbian principles. Rosenblatt introduced the perceptron learning rule, an iterative algorithm capable of minimizing error automatically. The system processed an input pattern (e.g., a pixelated character) and produced a binary classification. When the output deviated from the target, the algorithm adjusted the weights proportional to the error: strengthening connections that should have contributed to a correct firing and weakening those that led to false positives.

$$\sum_{i=0}^n x_i w_i \rightarrow \begin{cases} 1 & \text{if } \geq b \\ 0 & \text{if } < b \end{cases}$$

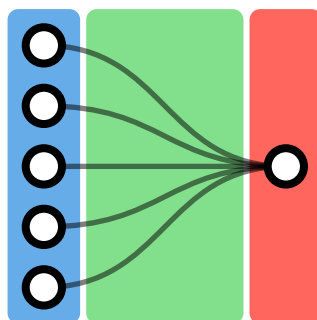


Figure 1: The perceptron model. Inputs x_i are multiplied by weights w_i and summed. If the linear combination $\sum x_i w_i$ exceeds the bias b , the neuron activates.

Consequently, the Perceptron was capable of converging on a solution for any problem where the data was linearly separable. This success generated significant enthusiasm, with contemporary reports suggesting that such machines would soon mimic human consciousness [1].

These expectations were abruptly tempered by theoretical limitations. In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, a rigorous mathematical analysis of the architecture. They demonstrated that a single-layer perceptron is fundamentally a linear classifier. While capable of learning operations like AND or OR, it is mathematically incapable of solving the XOR (Exclusive OR) problem. In the XOR case, the classes cannot be separated by a single hyperplane. This proof highlighted a severe boundary on the utility of single-layer networks for complex, non-linear tasks.

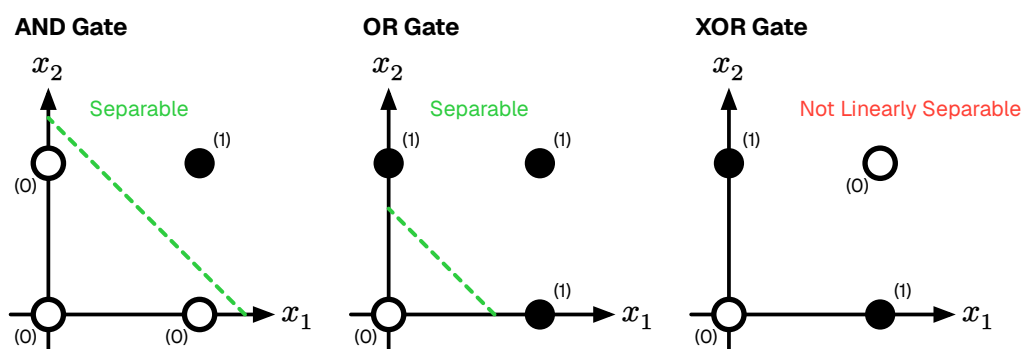


Figure 2: The XOR problem. Unlike AND/OR, the data points for XOR cannot be separated by a single linear boundary.

The publication of *Perceptrons* coincided with a significant reduction in neural network research funding, a period retrospectively termed the “First AI Winter”. It is worth noting that Minsky and Papert acknowledged that a Multi Layer Perceptron (MLP), a network stacking multiple layers of neurons, could theoretically solve the XOR problem by creating complex, non-linear decision boundaries.

However, a critical algorithmic gap remained: the “credit assignment problem”. While researchers knew that hidden layers could represent complex features, there was no known method to propagate error signals back through the layers to adjust the weights of hidden neurons effectively. Rosenblatt’s rule was mathematically valid only for the output layer. The field remained stagnant until a method for training multi-layer networks could be formalized.

2.2 • DEEP LEARNING

The critique presented by Minsky and Papert precipitated a contraction in funding; despite this, theoretical inquiry persisted. It was widely hypothesized that the limitations of the single perceptron could be overcome by a MLP. By organizing neurons (single perceptrons) into hierarchical layers, the network could theoretically perform successive non-linear transformations on the input space, enabling the formation of complex decision boundaries. The primary impediment was not the architecture itself, but the absence of a viable learning algorithm.

In a single-layer perceptron, error attribution is immediate: if the output deviates from the target, the error is directly derived from the weights of the output layer.

However, in a multi-layer architecture, quantifying the contribution of a specific neuron within the “hidden” layers to the final output error presents a significant challenge. This is formally known as the Credit Assignment Problem [1], and it remained the central theoretical obstacle for over a decade.

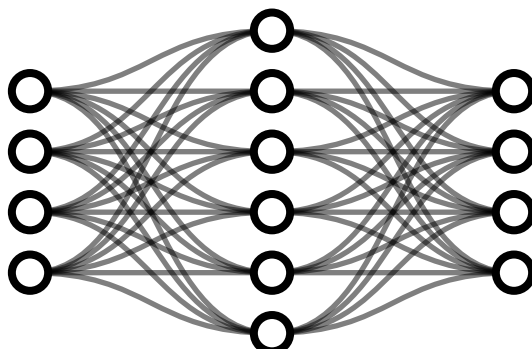


Figure 3: A MLP. By inserting “hidden layers” between input and output, the network can approximate non-linear functions such as XOR. The historical challenge lay in deriving a method to train these intermediate layers.

The solution to this theoretical impasse was popularized in 1986 by Rumelhart, Hinton, and Williams in their seminal paper *Learning representations by back-propagating errors* [1]. They demonstrated that the Chain Rule of calculus could be applied recursively to propagate the error signal from the output layer backwards through the hidden layers. This algorithm, known as Backpropagation, allowed the network to calculate the gradient of the loss function with respect to every weight in the system. Effectively, it provided a mathematical method to tell each hidden neuron exactly how much it contributed to the total error, finally solving the credit assignment problem.

Unlike Hebbian plasticity, which is local and biological, Backpropagation relies on global error signals and precise backward data flow—mechanisms effectively absent in organic tissue. Consequently, the field of Artificial Neural Network (ANN) effectively decoupled from neuroscience. It transitioned into a branch of engineering and applied mathematics, prioritizing statistical optimization over biological realism. Paradoxically, it was this abandonment of biological fidelity that enabled the rapid scaling and performance breakthroughs that followed.

ACHIEVEMENTS

With the training mechanism solved, the field exploded. The combination of Backpropagation, massive datasets, and GPU hardware led to a “Cambrian Explosion” of neural architectures, each solving domains previously thought impossible for computers.

The revolution began in earnest with computer vision. Convolutional Neural Networks (CNNs), such as AlexNet (2012) [1] and later ResNet [1], introduced the idea of learning hierarchical features—detecting edges, then shapes, then objects—much

like the human visual cortex. This allowed machines to classify images with super-human accuracy.

Soon after, the focus shifted to sequence data. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) architectures gave machines a short-term memory, enabling breakthroughs in speech recognition and machine translation. However, the true paradigm shift occurred with the introduction of the Transformer architecture in 2017. By utilizing an “attention mechanism” to parallelize the processing of language, Transformers allowed for the training of massive Large Language Models (LLMs) like the Generative Pre-trained Transformer (GPT).

These techniques have even transcended media generation. Deep Learning has solved fundamental scientific problems; notably, DeepMind’s AlphaFold utilized these architectures to predict the 3D structure of proteins from their amino acid sequences, a 50-year-old grand challenge in biology [1].

SHORTCOMINGS

Deep learning’s reliance on computational scaling masks fundamental inefficiencies in both its hardware implementation and underlying algorithms. By simulating biological concepts on digital architectures not designed for them, the current paradigm is approaching physical and economic limits.

A primary limitation is the Von Neumann architecture, which physically separates processing units from memory. Deep neural networks, defined by massive matrices of synaptic weights, necessitate constant data transfer. For every inference step, billions of parameters must be fetched from off-chip DRAM, processed, and written back. This creates a severe memory bottleneck where system performance is bounded by bandwidth rather than processing speed [1].

Consequently, the energy cost of moving data significantly exceeds the cost of computation itself. Retrieving a single byte from DRAM consumes approximately three orders of magnitude more energy than performing a floating-point operation [1]. Compounding this hardware friction, the dense matrix multiplications required for training scale quadratically with network size, making the pursuit of trillion-parameter models increasingly unsustainable.

Furthermore, the optimization algorithms driving this scale are fundamentally incompatible with physical biological systems. Backpropagation, while mathematically elegant, relies on a global error signal and suffers from the “weight transport problem”—the requirement that the backward pass utilizes the exact same synaptic weights as the forward pass. In organic tissue, synapses are unidirectional, and there is no known mechanism for a neuron to access the exact weight of a downstream synapse to calculate a gradient.

While a detailed technical analysis of these inefficiencies is presented in Section 4, the central issue is clear: modern AI prioritizes statistical optimization over physical

realism. Overcoming the limitations of the Von Neumann bottleneck and backpropagation requires a paradigm shift toward architectures that inherently co-locate memory and computation.

2.3 • BIRTH OF NEUROMORPHIC

While the artificial intelligence community debated symbolic logic versus connectionism during the “AI Winter,” significant developments were occurring in hardware physics. In the late 1980s at Caltech, physicist Carver Mead—a pioneer of Very Large Scale Integration (VLSI) design—began to question the trajectory of digital computing.

Mead observed that while digital computers were becoming exponentially faster, they were also becoming less efficient in terms of energy per operation. He noted that using transistors as rigid, high-power switches to perform boolean logic was energetically wasteful compared to the biological systems they aimed to emulate.

In 1990, Mead published his seminal paper, *Neuromorphic Electronic Systems* [1], coining the term “neuromorphic” to describe hardware that mimics the biological structure of the nervous system. His thesis proposed that rather than simulating neural equations via software on digital computers, engineers should construct physical hardware that exploits the same physical laws as the biological nervous system.

The foundational insight of the field was the physical analogy between silicon physics and ion-channel physics. In standard digital electronics, transistors are operated in “strong inversion,” driven by high voltages to act as binary switches. Mead realized that a single transistor, operating in its “subthreshold” region, follows the same exponential Boltzmann statistics that govern the flow of ions through biological channels.

This realization implied that a single transistor could physically compute the non-linear functions used by biological neurons, but with significantly higher speed and lower power consumption. Consequently, synaptic functions could be implemented by single transistors rather than complex arrangements of logic gates.

To demonstrate this concept, Mead and his doctoral student Misha Mahowald developed the *Silicon Retina* in 1991 [1]. Unlike a standard camera, which captures full frames at fixed intervals (generating redundant data), the Silicon Retina operated asynchronously. It utilized analog circuits to compute spatial and temporal derivatives directly on-chip, outputting discrete “events” only when local light intensity changed.

This event-driven approach solved the redundancy problem inherent in frame-based sampling. If the scene remained static, the system transmitted no data and consumed negligible energy. This demonstrated that by aligning the hardware

physics with the computational task, sensory information could be processed with a fraction of the power required by conventional digital systems.

Since the inception of neuromorphic computing, neuroscience has also advanced significantly. While Mead's early work was based on the physical intuition of the transistor, modern neuromorphic engineering now incorporates a richer understanding of neuronal dynamics, synaptic plasticity, and network architecture. To advance the field, we must combine these foundational hardware insights with the principles of modern mechanistic neuroscience.

3 • BIOLOGICAL PRINCIPLES

The biological brain remains the gold standard for energy-efficient, robust, and adaptive computation. Since the establishment of the Neuron Doctrine, modern neuroscience has uncovered the specific physical mechanisms that underpin this efficiency. To engineer systems that truly rival biological performance, we must transcend the “spherical cow” abstractions of early cybernetics. We cannot simply mimic the brain’s output; we must emulate its internal dynamics. This requires viewing the neuron not as a static summing unit, but as it functions in reality: a complex, time-dependent, and event-driven processor.

This section provides a mechanistic overview of the nervous system, translating biological observations into the computational primitives required for neuromorphic engineering. It explores the structural hierarchy of the neuron, the physics of the action potential, and the mathematical models used to capture these dynamics in silicon.

3.1 • NEURON STRUCTURE & FUNCTION

In Section 2 we established the neuron as the fundamental computational unit of the brain. While it shares standard cellular machinery like mitochondria and a nucleus with other cells, it is morphologically specialized for information transmission. A neuron consists of three functional zones:

- **The Input (Dendrites):** A branching tree structure that collects signals from thousands of upstream neurons. This is where inputs are integrated.
- **The Integration Zone (Soma):** The cell body where electrical potentials from the dendrites summate.
- **The Output (Axon):** A long, cable-like structure that transmits the neuron’s own signal to downstream targets.

The neuron exhibits a distinct morphological polarization that dictates the direction of information flow. The process begins at the “dendritic arbor”, a complex branching structure that maximizes the surface area for synaptic connectivity. These dendrites serve as the primary receptor sites, where neurotransmitters binding to post-synaptic terminals induce local conductance changes. These signals propagate passively toward the soma (cell body), the neuron’s central processing unit. The soma acts as an integrator, spatially and temporally summing the incoming synaptic currents. Finally, the processed signal is transmitted via the axon, a singular, elongated projection. In many vertebrate neurons, the axon is insulated by a myelin sheath, which facilitates saltatory conduction—a mechanism that allows high-speed signal propagation over long distances with minimal signal degradation.

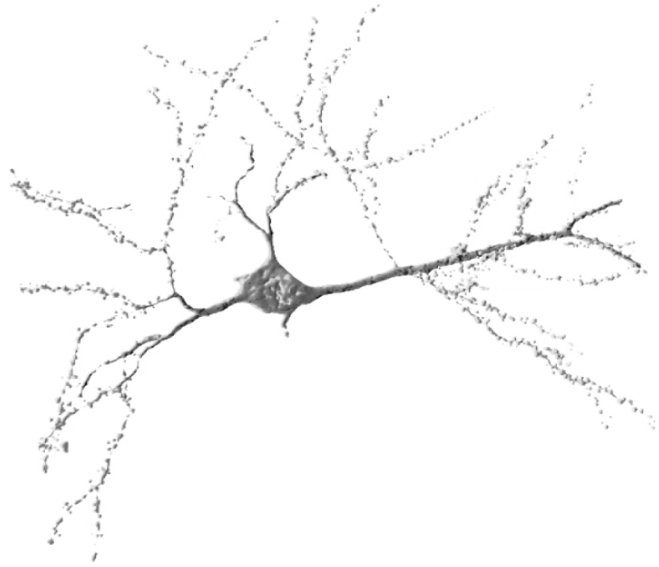


Figure 4: The morphological structure of a biological neuron, illustrating the directional flow of information from dendritic input to axonal output. © Angela Getz, Mathieu Ducros, Daniel Choquet / IINS et BIC / CNRS-Université de Bordeaux-Inserm.

Functionally, the neuron operates as an electrochemical system enclosed by a cell membrane, known as the “lipid bilayer”. This membrane is a thin, fatty structure that is impermeable to ions, acting as an electrical insulator. However, the fluids inside and outside the cell are conductive electrolytes. Consequently, the interaction between the insulating membrane and the conductive fluids creates a biological capacitor, capable of storing charge.

By actively pumping sodium (Na^+) out and potassium (K^+) in via the Na^+ - K^+ ATPase pump, the cell maintains an electrochemical gradient across this capacitor, resulting in a stable “resting potential” of approximately -70 mV.

Computation occurs through the modulation of this voltage by competing synaptic inputs. Excitatory inputs cause ion channels to open, allowing positive ions to influx; this reduces the negative charge (depolarization) and pushes the potential toward the firing threshold. Conversely, inhibitory inputs activate channels for negative ions (like Chloride, Cl^-), driving the potential away from the threshold (hyperpolarization). The soma integrates these opposing push and pull signals. If the aggregate membrane potential surpasses a critical threshold (approximately -55 mV), the system undergoes a bifurcating phase transition. Voltage-gated sodium channels cascade open, triggering an action potential—a rapid, non-linear depolarization spike that propagates down the axon. This mechanism is governed by the “all-or-nothing” principle: the output is discrete and binary, effectively filtering out sub-threshold noise.

Immediately following a spike, the neuron enters a “refractory period” during which ion gradients are restored, imposing a hard limit on the maximum firing frequency and ensuring the temporal separation of events.

It is important to acknowledge that the biological brain exhibits significant cellular diversity beyond this idealized model. The nervous system contains non-neuronal cells known as “glia”, which provide structural support and manage energy delivery, though they are generally not considered direct participants in fast information transmission. Additionally, while the vast majority of cortical neurons communicate via uniform action potentials (spikes), certain sensory neurons utilize “graded potentials”, where the signal amplitude varies continuously. However, as spiking neurons represent the dominant computational paradigm for information processing in the cortex, this thesis focuses exclusively on the spiking model as the basis for neuro-morphic emulation.

3.2 • ACTION POTENTIAL & SPIKE TRAINS

As established in the previous section, the action potential is an “all-or-nothing” event. It serves as the fundamental mechanism by which neurons transmit information. Crucially, the waveform of this event is stereotypical: for a given neuron, every spike exhibits a nearly identical amplitude and duration (typically 1–2 ms), independent of the input intensity that triggered it.

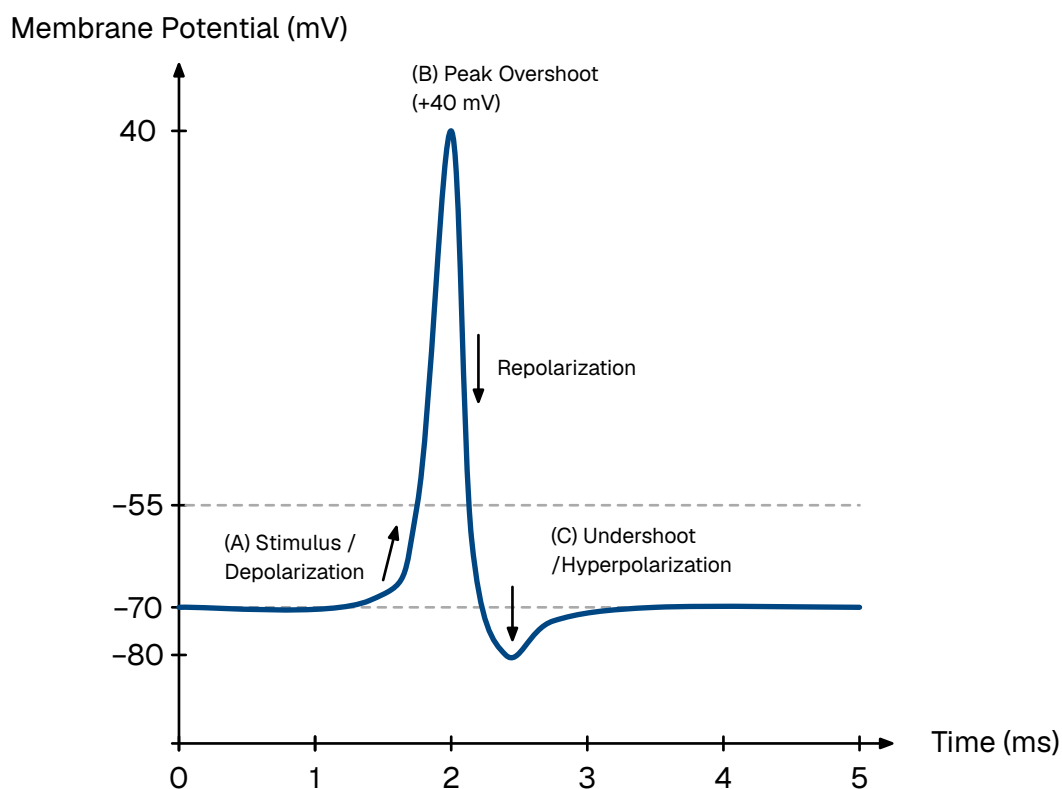


Figure 5: The phases of a typical neuronal action potential. (A) An incoming stimulus depolarizes the membrane past the threshold (-55 mV), triggering a rapid spike. (B) The membrane potential reaches a peak overshoot ($+30$ mV) before repolarizing. (C) A temporary undershoot (hyperpolarization) occurs before returning to the resting state (-70 mV). The neuron cannot fire during the absolute refractory period (D) and requires a stronger stimulus to fire during the relative refractory period (E).

This biological invariance permits a fundamental simplification in neuromorphic modeling:

If the spike waveform is invariant across neurons and time, the waveform itself carries no information. Consequently, the information content of the signal is encoded entirely in the precise timing of the spike.

To model this mathematically, we abstract the continuous biophysical voltage trace into a dimensionless point process. We treat the action potential not as a function of voltage over time, but as a singular event occurring at a precise instant, t_f , with negligible duration. The standard tool for this abstraction is the Dirac delta function denoted as, $\delta(t)$.

The Dirac delta is a generalized distribution defined by the property that it is zero everywhere except at the origin, yet integrates to unity. This represents an idealized pulse of infinite height and zero width, containing a finite unit of effect.

$$\delta(t) = \begin{cases} \infty & \text{if } t = 0 \\ 0 & \text{if } t \neq 0 \end{cases}, \quad \int_{-\infty}^{+\infty} \delta(t) dt = 1$$

Equation 1: The defining properties of the Dirac delta function.

Under this formalism, the output of a neuron is modeled not as a continuous signal, but as a “spike train”—a temporal sequence of these Dirac impulses. For a neuron emitting N spikes at times $\{t^{(1)}, t^{(2)}, \dots, t^{(N)}\}$, the output signal $S(t)$ is defined as:

$$S(t) = \sum_{f=1}^N \delta(t - t^{(f)})$$

Equation 2: A spike train represented as a sum of Dirac delta functions.

This abstraction allows the post-synaptic effect to be modeled using linear systems theory. In neuron models that use this framework, the interaction is treated as instantaneous charge deposition: the arrival of a delta function $\delta(t - t_f)$ imparts a discrete step-change to the post-synaptic current. This mimics the rapid opening of ion channels without requiring the computational overhead of simulating the complex voltage trajectory. The shift from continuous values to discrete spike trains fundamentally alters the computational paradigm, moving from spatial representations (magnitude-based) to spatio-temporal representations (time-based).



Figure 6: Transformation of continuous membrane voltage (top) into a discrete spike train (bottom).

3.3 • NEURON MODELS

In the quest to simulate the brain, there exists a fundamental trade-off between biological realism and computational efficiency. At the high end of the spectrum lie conductance-based models, most notably the Hodgkin-Huxley model. This formalism describes the neuron not as a simple computational unit, but as an electrical circuit with variable resistors representing the precise, non-linear opening and closing dynamics of specific ion channels (sodium, potassium, leak) [1].

Large-scale initiatives, such as the Blue Brain Project, utilize even more granular “multi-compartment” models. These simulations treat the neuron as a complex 3D structure, discretizing the dendritic arbor and axon into hundreds of segments to model how current flows through the specific morphology of the cell [1]. While invaluable for pharmacological research, these models are computationally prohibitive for large-scale neuromorphic engineering. Simulating a mere second of biological time for a small network using these equations requires supercomputing resources.

To build practical, scalable neuromorphic hardware, we must abstract these biophysical details into a phenomenological model. We seek a mathematical framework that captures the essential computational properties—integration, leakage, and thresholding—without simulating the underlying molecular physics.

THE LEAKY INTEGRATE-AND-FIRE (LIF) MODEL

The standard approximation used in neuromorphic engineering is the Leaky Integrate and Fire (LIF) model. This framework aligns perfectly with the “point process” abstraction established in the previous section, as it treats action potentials as instantaneous, discrete events. Its state is defined by a single scalar variable, the membrane potential $u(t)$. The sub-threshold dynamics are governed by a linear differential equation analogous to a simple RC (Resistor-Capacitor) circuit:

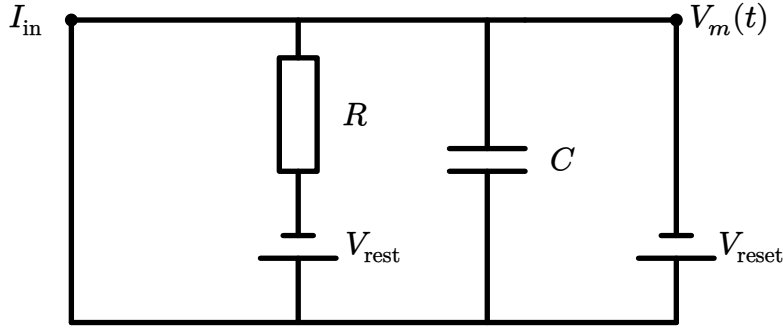


Figure 7:

$$\tau_m \frac{du}{dt} = -(u - u_{rest}) + RI(t)$$

Equation 3: The Leaky Integrate-and-Fire (LIF) differential equation. The change in voltage is driven by the leak (decay to rest) and the input current.

Where τ_m is the membrane time constant (determining how fast the neuron “forgets”), u_{rest} is the resting potential, R is the membrane resistance, and $I(t)$ is the input current.

Connecting this to the spike train abstraction derived in the previous section, the input current $I(t)$ is not continuous. It is a sequence of discrete events arriving from pre-synaptic neurons j with weight w_j . Mathematically, this is modeled as a sum of Dirac delta functions:

$$I(t) = \sum_j jw_j \sum f\delta(t - t_{j(f)})$$

Equation 4: Synaptic input modeled as a weighted sum of Dirac delta functions.

Because the differential equation is linear below the threshold, we can solve it analytically. The membrane potential $u(t)$ becomes a convolution of the input spike train with the system’s impulse response (a decaying exponential kernel). This means the potential at any moment is simply the sum of the decaying traces of all past spikes:

$$u(t) = u_{\text{rest}} + \sum jw_j \sum f \exp\left(-\frac{t - t_{j(f)}}{\tau_m}\right)$$

Equation 5: The analytical solution for the membrane potential. The current voltage is the superposition of all past inputs, decayed by time constant τ_m .

The differential equation above describes the continuous sub-threshold dynamics. To complete the model, we must define the discrete “Fire” mechanism. The LIF neuron operates as a hybrid dynamical system: it integrates continuously until a discontinuity is triggered.

When the membrane potential $u(t)$ exceeds a specific threshold voltage ϑ , the neuron emits a spike (a Dirac delta event). Immediately following this event, the membrane potential is not governed by the differential equation but is forced to a reset value, u_{reset} .

To mimic the biological limit on firing frequency, the model enforces an absolute refractory period, Δ_{ref} . After a spike occurs at time t_f , the neuron is clamped to the resting potential for a duration of Δ_{ref} . During this interval, the differential equation is suspended; the neuron ignores all inputs and cannot fire, regardless of the stimulus intensity.

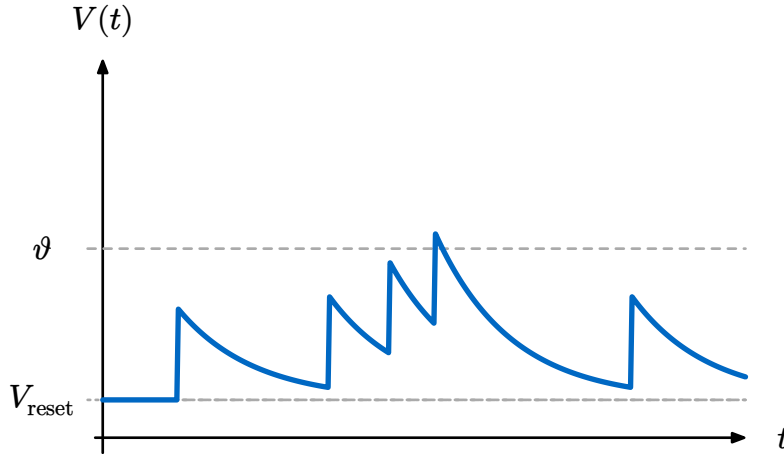


Figure 8: The dynamics of an LIF neuron. (A) The membrane potential integrates inputs. (B) Upon crossing the threshold ϑ , a spike is emitted and the voltage is reset. (C) The voltage is clamped during the refractory period.

Mathematically, this firing condition is expressed as:

$$\text{If } u(t) > \vartheta \rightarrow \begin{cases} \text{Emit spike: } S(t) \rightarrow S(t) + \delta(t) \\ \text{Reset voltage: } u(t) \rightarrow u_{\text{reset}} \\ \text{Pause integration for } t \in (t_f, t_f + \Delta_{\text{ref}}] \end{cases}$$

Equation 6: The discrete firing and reset condition.

This equation represents the engine of most neuromorphic algorithms. It defines a system that integrates information over time and leaks it to ensure temporal rele-

vance. However, once $u(t)$ crosses a threshold ϑ , the linearity breaks and the neuron emits a spike and $u(t)$ is manually reset to u_{reset} .

THE GENERALIZED (ADAPTIVE) LIF MODEL

While the standard LIF model is computationally efficient, its one-dimensional nature limits it primarily to tonic spiking (regular firing under constant input). It struggles to replicate the complex, non-linear behaviors observed in the cortex, such as bursting (clusters of rapid spikes) or spike-frequency adaptation (slowing down after sustained activity).

To capture these dynamics without reverting to the computationally heavy Hodgkin-Huxley equations, we employ the Generalized Leaky Integrate and Fire (GLIF) model. This extends the system by introducing a second state variable, $w(t)$, representing cellular adaptation.

$$\begin{aligned}\tau_m \frac{du}{dt} &= -(u - u_{\text{rest}}) + RI(t) - w \\ \tau_w \frac{dw}{dt} &= a(u - u_{\text{rest}}) - w\end{aligned}$$

Equation 7: The Adaptive GLIF system. The adaptation variable w provides negative feedback, enabling complex dynamics like bursting and adaptation.

In this coupled system, w provides a negative feedback loop. Every time the neuron spikes, w increments by a constant b , acting as a physiological drag on the membrane potential. By adjusting the coupling parameters between u and w , this two-dimensional system can be tuned to emulate the full spectrum of biological firing patterns.

It is natural to question whether such a mathematically reduced model can genuinely capture the behavior of biological neurons. While the GLIF model discards the specific ionic mechanisms of the Hodgkin-Huxley equations, empirical validation demonstrates that it retains superior computational dynamics for large-scale modeling.

In the 2008 *Quantitative Single-Neuron Modeling Competition* [1] organized by the INCF, phenomenological models like the Generalized LIF (specifically the Adaptive Exponential Integrate-and-Fire) unexpectedly outperformed highly detailed biophysical models in predicting the precise spike times of real cortical neurons.

This counter-intuitive success is due to parameter sensitivity. Complex conductance-based models have dozens of unobservable parameters that are difficult to tune. In contrast, the GLIF model captures the “net effect” of these mechanisms using macroscopic parameters that can be robustly fitted to data. As demonstrated by Izhikevich (2003), this simple system of two differential equations is capable of reproducing all known firing patterns observed in the mammalian cortex [1].

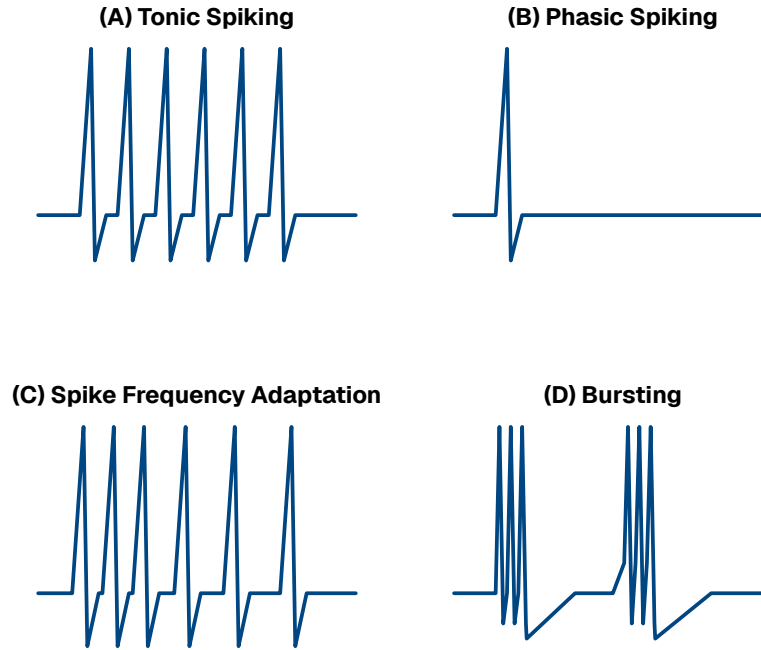


Figure 9: The Generalized LIF model is capable of reproducing the diverse firing patterns of biological cortical neurons, as categorized by Izhikevich (2003) [1].

Consequently, for the purpose of neuromorphic engineering, the GLIF model represents the optimal trade-off between biological fidelity and computational efficiency.

3.4 • NEURAL CODING

In classical digital computing, information is represented by combining bits into richer structures, such as floating-point or integer numbers. For instance, the luminance of a pixel is typically stored as a discrete 8-bit or 32-bit integer. Conversely, analog electronics represent values as continuous currents or voltages, offering infinite resolution within the dynamic range of the hardware.

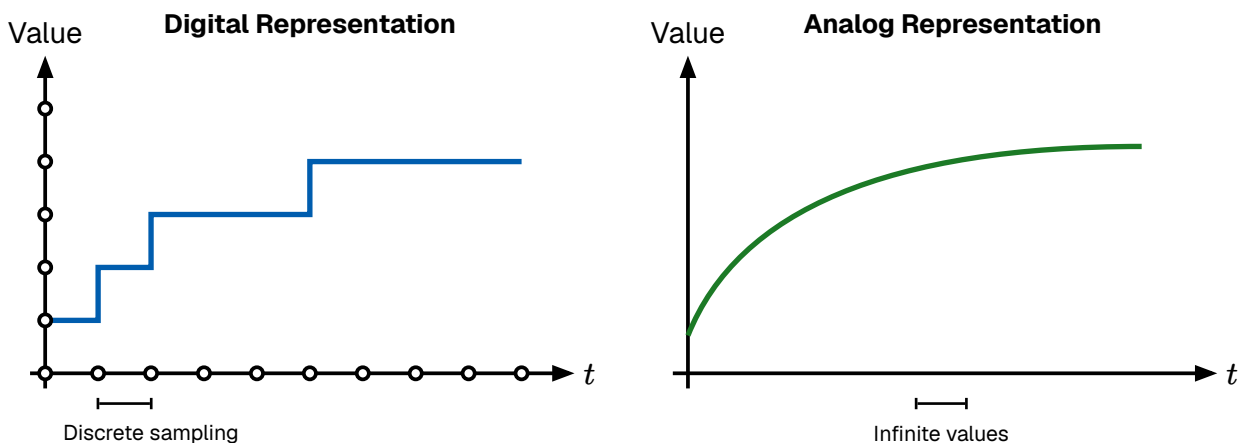


Figure 10: Digital left analog right representation

The biological brain occupies a unique middle ground. While neurons operate using analog membrane potentials, their communication output—the action potential—is discrete and binary. As established in Section 3.2, the waveform of a spike is stereo-

typical; it looks like a “digital bit” in amplitude. However, unlike a digital computer which is synchronized to a rigid clock, these spikes occur in continuous time. Therefore, the information in the nervous system is not stored in the shape of the signal, but in the structure of the spike train itself.

Deciphering the “Neural Code”—the set of rules by which sensory stimuli are translated into these spike sequences—remains one of the central challenges in neuroscience. Currently, several coding schemes are hypothesized to coexist, each offering different trade-offs between latency, information density, and robustness.

RATE CODING

The most traditional interpretation of neural activity is rate coding. In this paradigm, information is conveyed by the mean firing frequency of a neuron over a specific temporal window. A strong stimulus (e.g., high pressure on skin) elicits a high firing rate, while a weak stimulus results in sparse activity.

This model effectively treats the neuron as an Analog-to-Digital converter where the precise timing of individual spikes is treated as noise; only the average count carries the signal. While rate coding is robust and easily observed in motor neurons, it suffers from a fundamental latency barrier. To estimate a rate with reasonable precision, the post-synaptic neuron must integrate spikes over a significant duration (tens or hundreds of milliseconds). This contradicts the rapid reaction times (often < 100 ms) observed in biological agents, suggesting that rate coding alone cannot account for time-critical processing.

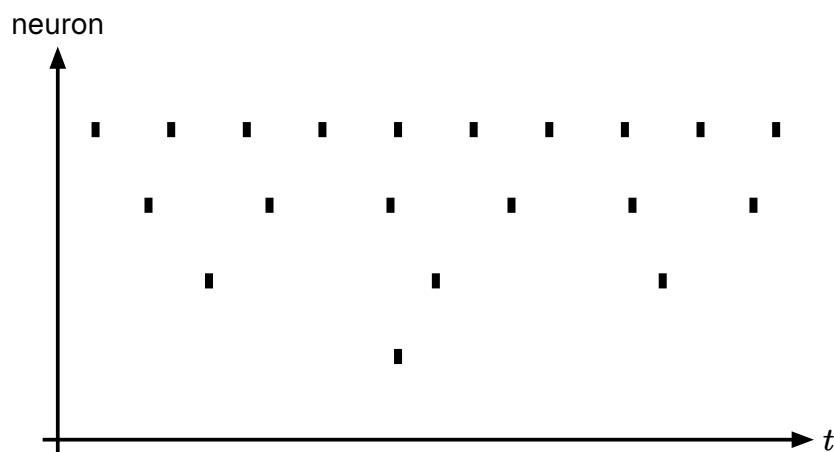


Figure 11: Rate Coding: The stimulus intensity is encoded in the frequency of the spike train. Stronger stimuli elicit more spikes per second.

TEMPORAL CODING

To explain the speed of biological processing, neuromorphic engineering emphasizes temporal coding. In this regime, the precise timing of a spike carries significant information. A primary example is Time-To-First-Spike (TTFS) coding.

In a TTFS scheme, the intensity of a stimulus is inversely mapped to the latency of the response relative to a stimulus onset. A stronger input causes the neuron to integrate and cross the threshold faster, firing earlier than neurons receiving weak inputs. This shifts the computational model from counting spikes to a “race” between spikes.

In a network utilizing lateral inhibition (Winner-Take-All (WTA)), the first neuron to fire inhibits its neighbors, allowing a decision to be made as soon as the first meaningful bit of data arrives. This eliminates the need to wait for a time window to close, drastically reducing latency. Furthermore, since TTFS coding prioritizes the strongest signals, it acts as a natural filter: the most prominent features arrive first, allowing the system to process signal over noise.

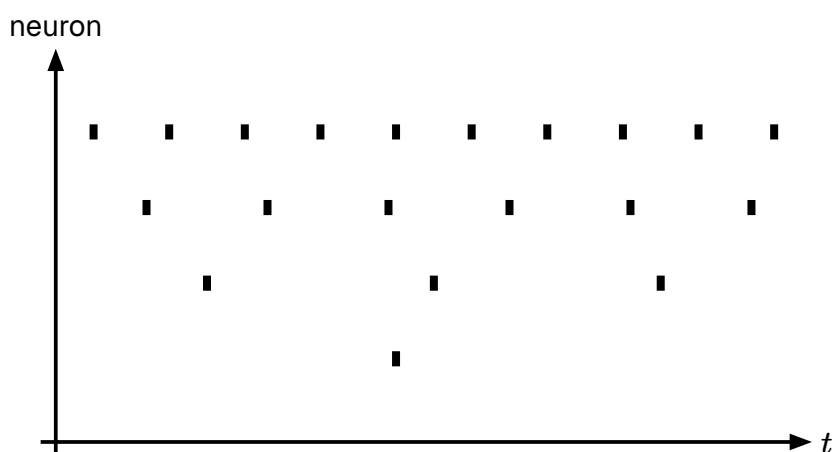


Figure 12: Temporal Coding (TTFS): Stimulus intensity is encoded in the latency of the response. Stronger inputs (I_1) trigger an earlier spike (t_1) compared to weaker inputs (I_2).

THE PHASE AMBIGUITY PROBLEM

A critical challenge in temporal coding is the need for a temporal reference frame. In Rate Coding, the “phase” (absolute timing) is irrelevant. However, in Temporal Coding, a spike at time t only has meaning relative to a reference t_0 . If the receiver does not know when the stimulus started, it cannot decode the latency.

In engineering, this is solved by a global clock or a “frame start” signal. In the brain, evidence suggests that background oscillatory rhythms (brain waves, such as theta or gamma cycles) may serve as this global reference, allowing populations of neurons to synchronize their “clocks” and decode relative timings accurately.

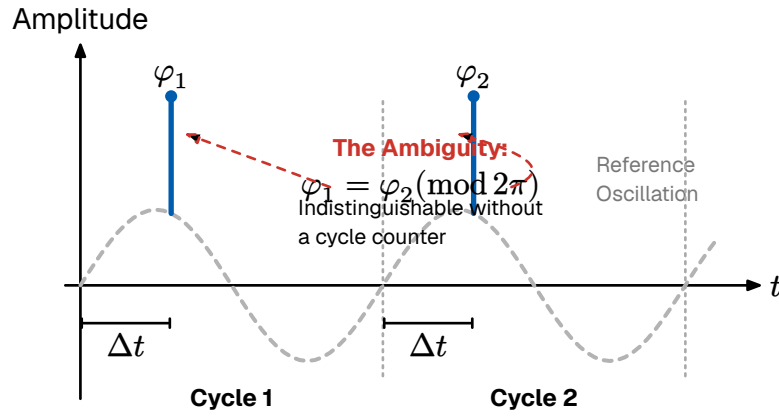


Figure 13: The phase ambiguity problem in temporal encoding. Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\text{mod } 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

POPULATION & SPARSE CODING

While single-neuron codes provide the basic signaling mechanism, the brain employs ensemble strategies to ensure robustness and precision. In population coding, variables are represented by the joint activity of a large group of neurons, each with broad, overlapping tuning curves. A classic example is found in the Primary Visual Cortex (V1), where orientation-selective neurons each respond maximally to a preferred angle but also fire weakly for nearby orientations. By reading the weighted population vector across the group, the network reconstructs the stimulus with far greater precision than any individual cell could provide alone. The brain further optimizes for metabolic efficiency through sparse coding, where only a small fraction of neurons are active at any moment. This strikes a mathematical balance between representational capacity and energy cost, and is naturally enforced by lateral inhibition circuits that suppress weaker, competing responses.

COEXISTENCE OF CODES

These schemes are not mutually exclusive but complementary. A circuit may use TTFS for a rapid initial response — alerting the system to a salient change — before transitioning to rate-based activity for sustained processing. Neuromorphic systems often adopt this hybrid approach, using temporal codes for energy-efficient sparse event transmission and rate-based readouts for interfacing with downstream control systems. This thesis follows the same principle, using TTFS encoding for the transmission of visual features combined with a population-level representation at the hidden layer.

3.5 • NEURAL NETWORKS

Having established the mathematical description of the individual neuron, we now turn to the collective behavior of these units. A single neuron, regardless of its dynamical complexity, is of limited computational utility in isolation. Functional intelligence emerges only when these units are organized into specific structural topologies.

The brain is not a random mesh of connections; it is constructed from recurring architectural “motifs” that appear across various cortical areas. Understanding these motifs is essential for designing neuromorphic systems that transcend simple feed-forward processing.

SYNAPTIC EFFICACY & WEIGHTS

Before analyzing the structural topology of networks, we must define the fundamental unit of connectivity: the synapse. In the biological brain, neurons do not touch; they are separated by a microscopic gap known as the synaptic cleft. Communication across this gap is chemical, mediated by the release of neurotransmitters.

The efficiency of this transmission—determined by factors such as the amount of neurotransmitter released and the number of post-synaptic receptors—is abstracted in mathematical models as the synaptic weight (w).

In the Spiking Neural Network (SNN) formalism, the weight represents a scaling factor for the incoming spike. When a pre-synaptic neuron j fires a spike at time t_j , it induces a Post Synaptic Current (PSC) in neuron i scaled by the weight w_{ij} . Mathematically, the total synaptic input $I(t)$ is the weighted sum of all incoming spike trains:

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Equation 8: The synaptic input current as a weighted sum of incoming impulses.

Synaptic weights determine not just the magnitude but also if the synapse is excitatory or inhibitory.

- **Excitatory Synapses ($w > 0$):** These depolarize the target neuron, pushing its membrane potential closer to the firing threshold (e.g., Glutamate synapses).
- **Inhibitory Synapses ($w < 0$):** These hyperpolarize the target neuron, pushing the potential away from the threshold (e.g., GABA synapses).

A fundamental constraint in biological networks, known as Dale’s Principle, states that a neuron performs the same chemical action at all of its synaptic outputs. This means a neuron is strictly excitatory or strictly inhibitory; it cannot send positive

signals to one neighbor and negative signals to another. While standard ANNs often violate this rule for mathematical convenience (allowing weights to flip signs during training), bio-plausible neuromorphic architectures often enforce this constraint to mimic the distinct populations of Pyramidal (excitatory) and Interneuron (inhibitory) cells found in the cortex.

The network must maintain a precise Excitation-Inhibition (E/I) Balance. The brain operates at a critical point of instability:

- **Excess Excitation** leads to runaway feedback loops (analogous to epileptic seizures).
- **Excess Inhibition** leads to signal extinction (quiescence).

DIRECTIONALITY

Structurally, neural topologies can be categorized by the flow of information.

In sensory peripheries (such as the retina) and early processing stages, information flows unidirectionally from input to output. This topology supports rapid, reflex-like feature extraction. This configuration is known as a feed-forward network, which is mathematically equivalent to a Directed Acyclic Graph (Directional Acyclic Graph (DAG)) and serves as the standard architecture for most Deep Learning CNNs.

In higher cognitive areas, the dominant topology is recurrence. Neurons form feedback loops, connecting back to themselves or to distinct layers. This recurrence introduces a time component to the computation, transforming the network into a dynamical system where the current output depends not only on the input but on the network's previous state (history).

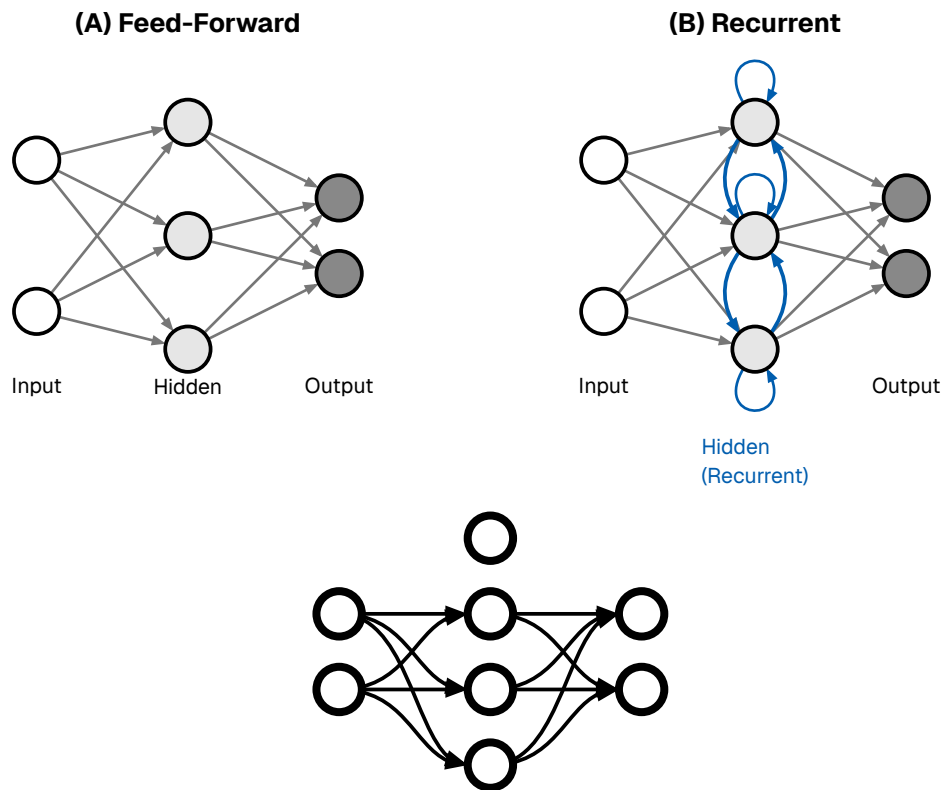


Figure 14: Network topologies. (A) Feed-Forward. (B) Recurrent.

THE SYNAPTIC HYPOTHESIS: STRUCTURE AS FUNCTION

A foundational premise in neuromorphic engineering, derived from biological observation, is that the neuron operates largely as a generic processing unit. The functional identity of a neural circuit—whether it processes visual stimuli or governs motor control—is determined principally by the topology and efficacy of its synaptic interconnections.

This paradigm, known as the Synaptic Hypothesis, posits that the physical configuration of synaptic weights constitutes the substrate for all computation and memory. Unlike traditional Von Neumann architectures, where data is retrieved from a distinct memory module and processed in a central CPU, biological systems eliminate the distinction between “data” and “program.” Memory is not a static artifact, but a latent computational potential distributed across the network’s structural graph. Consequently, learning in a neuromorphic system is realized through the physical alteration of these synaptic weights, ensuring robust, decentralized processing that is inherently resistant to localized hardware failure (graceful degradation).

INHIBITION PATTERNS

A ubiquitous micro-circuit motif in the cortex is lateral inhibition. In this configuration, an active excitatory neuron stimulates distinct inhibitory interneurons, which in turn suppress the activity of neighboring excitatory neurons. This competition engenders a WTA dynamic: as one neuron—representing a specific feature or decision—becomes active, it effectively silences its competitors. In the context of

neuromorphic engineering, WTA circuits are indispensable; they provide a physical mechanism for both noise reduction, by actively suppressing weak, sub-threshold signals, and categorical decision making, enabling the circuit to autonomously select the most salient option without the need for a central processor to sort or compare values.

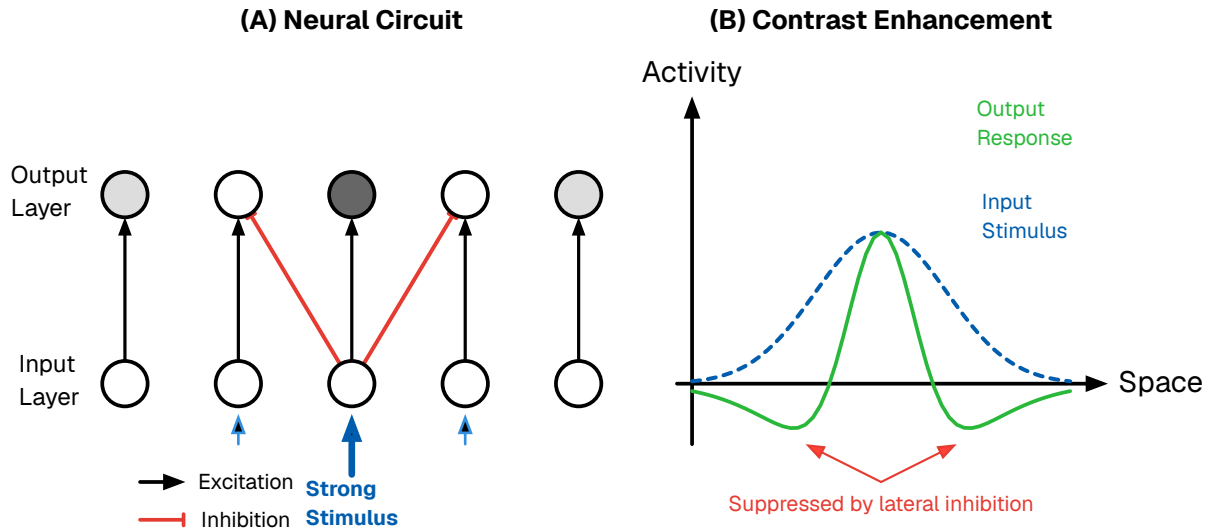


Figure 15: The mechanism of lateral inhibition. (A) A highly stimulated neuron in the input layer strongly excites its corresponding output neuron while simultaneously sending lateral inhibitory signals to its immediate neighbors. (B) This architectural motif acts as a spatial filter, producing a contrast enhancement effect. A broad input stimulus (dashed blue line) is transformed into a sharper output response (solid purple line) characterized by an amplified center and suppressed surroundings (a “Mexican hat” profile), thereby sharpening signal boundaries.

While lateral inhibition processes information in the spatial domain, Feed-Forward Inhibition (FFI) operates in the temporal domain. Structurally, this motif bifurcates an input signal into two parallel pathways: a direct excitatory route to the target neuron, and a disynaptic inhibitory route that reaches the same target with a slight synaptic delay. This architecture creates a narrow “temporal window of opportunity.” Because the excitation triggers the neuron immediately before the delayed inhibition abruptly truncates the response, the neuron is prevented from integrating noise over extended durations. Consequently, FFI forces the neuron to function as a precise Coincidence Detector rather than a sluggish integrator, a dynamic that is fundamental to sound localization in the auditory cortex and fine-grain timing in the somatosensory system.

3.6 • BIOLOGICAL LEARNING

As previously established, the functional identity of a neural circuit is not defined by a transient software state, but by its physical hardware configuration. Consequently, “learning” in a biological substrate cannot be viewed as a simple parameter optimization; it is a fundamental morphological process. If structure dictates function,

then the acquisition of new skills or memories necessitates the physical restructuring of the connectome itself.

Because the brain lacks a central supervisor or global communication bus, this restructuring must be driven by Locality. A synapse can only change based on information physically available at the cleft: the activity of the pre-synaptic axon, the voltage of the post-synaptic dendrite, and the immediate neurochemical environment. Despite this constraint, the brain successfully credits specific synaptic events with outcomes that occur seconds or minutes later.

This adaptation occurs across multiple timescales and spatial resolutions via two distinct mechanisms: Structural Plasticity (the rewiring of the network topology) and Synaptic Plasticity (the modulation of connection strength).

STRUCTURAL PLASTICITY

While synaptic weight adjustment accounts for rapid learning and pattern recognition, the long-term storage of information and the optimization of energy efficiency are governed by structural plasticity. This mechanism involves the physical genesis (synaptogenesis) and removal (pruning) of synapses and even entire axonal branches.

- **Synaptogenesis:** When neurons are repeatedly co-active but lack a direct connection, the brain can physically grow new dendritic spines and axonal boutons to bridge the gap. This effectively alters the network's topology, creating new pathways for information flow where none existed before.
- **Pruning:** Equally critical is the removal of redundant or noisy connections. During sleep and developmental critical periods, the brain aggressively prunes weak synapses. This “sparsification” reduces metabolic cost and increases the signal-to-noise ratio of the circuit by removing irrelevant pathways.

In the context of the Synaptic Hypothesis, structural plasticity represents the “compiling” of temporary associations into permanent hardware architecture.

SYNAPTIC PLASTICITY

Once a structural connection exists, its efficacy—the magnitude of the post-synaptic response to a pre-synaptic spike—must be tuned. In biological terms, this “weight” corresponds to the amount of neurotransmitter released and the density of receptors on the receiving side. This fine-grained adjustment is governed by local learning rules.

HEBBIAN LEARNING: RATE-BASED CORRELATION

The foundational axiom of biological learning was postulated by Donald Hebb in 1949. Hebb proposed that synaptic efficiency is a function of the correlated activity between two neurons. Colloquially summarized as “Neurons that fire together, wire

together,” this rule implies that the brain learns by detecting statistical regularities in sensory input.

Mathematically, if neuron A consistently takes part in firing neuron B , the connection from A to B is strengthened. This mechanism allows the brain to perform unsupervised clustering, physically encoding associations between features that occur simultaneously in the environment (e.g., the smell of smoke and the sight of fire).

SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Modern neuroscience has refined Hebb’s macroscopic theory into a precise, millisecond-scale mechanism known as Spike-Timing-Dependent Plasticity (STDP). Unlike rate-based models, STDP operates on the precise timing of individual action potentials, introducing the critical element of causality.

The STDP rule adjusts the synaptic weight based on the relative timing difference (Δt) between the pre-synaptic input and the post-synaptic output:

- **Long-Term Potentiation (LTP):** If the input spike arrives **before** the output spike ($\Delta t > 0$), it implies the input contributed to the firing. The synapse is strengthened to reinforce this causal link.
- **Long-Term Depression (LTD):** If the input spike arrives **after** the output spike ($\Delta t < 0$), the input was irrelevant to the decision. The synapse is weakened.

This asymmetry allows the network to self-organize, naturally filtering out random noise while reinforcing specific spatiotemporal patterns.

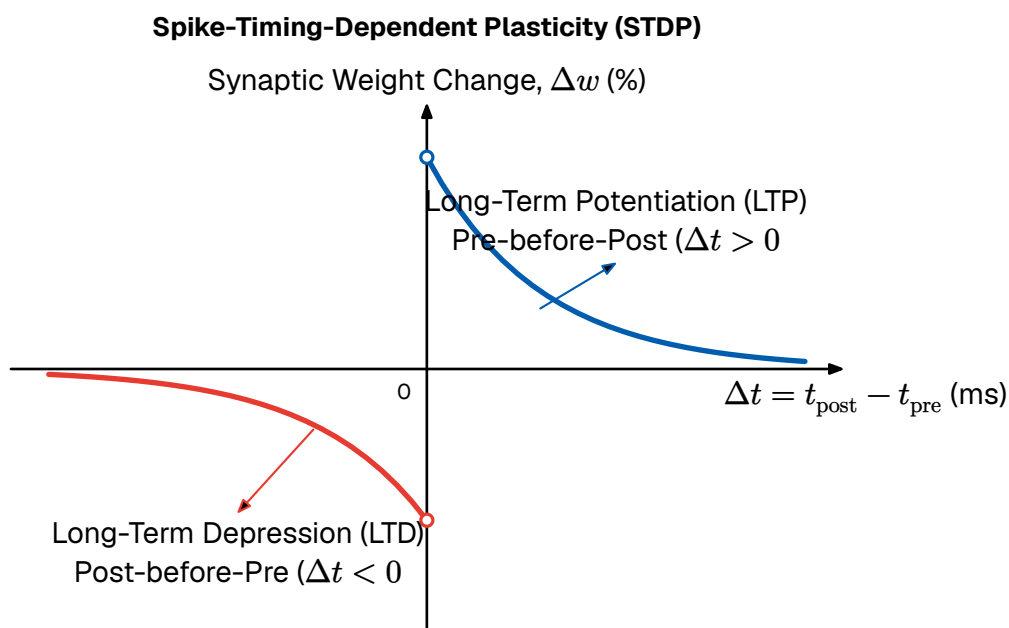


Figure 16: The STDP Learning Curve. Synaptic weight change is plotted against spike timing difference. Pre-before-post timing triggers strengthening (LTP), while post-before-pre triggers weakening (LTD).

HOMEOSTATIC PLASTICITY

If Hebbian mechanisms (LTP) were the sole drivers of plasticity, neural networks would be inherently unstable. A positive feedback loop would emerge where strengthened synapses drive higher firing rates, which in turn induce further strengthening, leading to runaway excitation (seizures). Conversely, unchecked LTD could silence a network entirely.

To maintain stability, the brain employs Homeostatic Plasticity (or Synaptic Scaling). This is a global regulatory mechanism that operates on a slower timescale (minutes to hours). It functions as a negative feedback loop: if a neuron's average firing rate exceeds a target set-point, the cell chemically downscales the strength of all its incoming synapses. This ensures that neurons remain within a sensitive dynamic range, preventing saturation regardless of how strong the inputs become.

4 • TECHNICAL DETAILS OF MACHINE LEARNING

This chapter delineates the technical foundations of modern artificial intelligence, contrasting the established paradigms of Deep Learning (DL) with the emerging principles of Neuromorphic Engineering. We begin by analyzing the algorithmic architecture of standard Deep Learning, identifying the computational bottlenecks inherent in its reliance on dense matrix multiplication and backpropagation.

A critical distinction must be drawn between biological plausibility and bio-inspired engineering. From an engineering perspective, the primary objective is functional utility. An engineer may treat the brain merely as a source of heuristic inspiration rather than a blueprint to be copied dogmatically. However, the pursuit of biologically plausible systems remains vital; it offers potential advantages in robustness and energy efficiency while serving as a verification tool for neuroscience.

4.1 • OPTIMIZATION

Optimization is the selection of a “best candidate” with regard to defined criteria. Biological learning fits this description, where the optimal candidate is the configuration of synaptic weights that performs well for a specific task. Therefore, it is useful to establish a mathematical framework for this process.

Fundamentally, a deep learning model operates as a function approximator. We assume the existence of an unknown underlying function $f : X \rightarrow Y$ that perfectly maps inputs to their target outputs. Since this true function is unknown, we construct a family of hypothesis functions $f_{\theta}(\mathbf{x})$ to approximate it. Here, $\theta \in \mathbb{R}^d$ represents the state of the system—a vector containing all tunable parameters, such as synaptic weights or biases. The dimensionality d corresponds to the degrees of freedom of the model.

The key problems in optimization are defining the objective goal (the loss function) and finding the parameter configuration θ that achieves that goal.

SUPERVISED LEARNING

To guide the search for optimal parameters $\hat{\theta}$, we must quantify the divergence between the model’s predictions and the ground truth. We define a scalar Loss Function $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ that evaluates the error on a single data point. To ensure generalization, we seek to minimize the Empirical Cost Function $J(\theta)$, defined as the average loss over a dataset of size N :

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Geometrically, the cost function $J(\boldsymbol{\theta})$ induces an Optimization Landscape. Finding a low-energy state in this non-convex topology is the central challenge of AI training. We rely on iterative optimization algorithms, principally Gradient Descent. This method updates the system state in the direction opposite to the gradient vector $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ (the steepest ascent). The update rule for iteration t is:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

Here, η represents the Learning Rate. Because computing the gradient over the entire dataset N is computationally prohibitive, modern AI employs Stochastic Gradient Descent (SGD), approximating the gradient using small random subsets (mini-batches). This introduces beneficial noise, preventing the system from getting trapped in shallow local minima.

Crucially, gradient descent requires the loss function to be differentiable. As will be discussed later, this presents a significant challenge for optimizing neuromorphic systems utilizing discrete, non-differentiable spike trains.

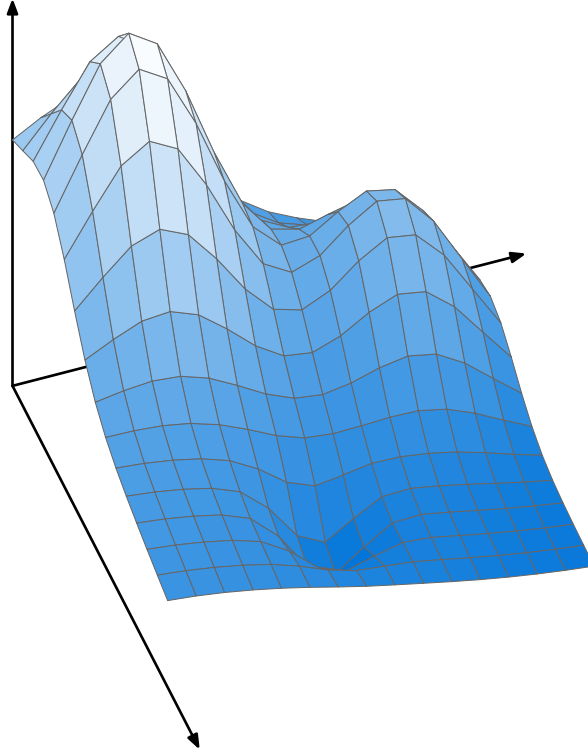


Figure 17: The Optimization Landscape. The system seeks to traverse the high-dimensional surface defined by $J(\boldsymbol{\theta})$ to find the global minimum $\boldsymbol{\theta}^*$, using the gradient ∇J as a navigational compass.

Strictly minimizing the empirical cost carries the risk of overfitting — the model memorizes training data including noise rather than learning the underlying function. In biological systems this is naturally regulated by metabolic constraints; the

brain prunes weak connections to maintain a sparse topology, effectively trading model complexity for generalization. In artificial systems this is managed via explicit regularization penalties added to the cost function.

UNSUPERVISED LEARNING

While supervised learning relies on labeled targets, biological systems predominantly learn via Unsupervised Learning. In this regime, the dataset consists only of input vectors $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$. The optimization objective shifts from minimizing prediction error to minimizing representation error.

Mathematically, the goal is often to discover a lower-dimensional manifold that efficiently captures the structure of the data. A common formulation is the minimization of Reconstruction Loss, where the system attempts to compress the input into a latent code and reconstruct it:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - f_{\text{decode}}(f_{\text{encode}}(\mathbf{x}_i; \boldsymbol{\theta}))\|^2$$

Alternatively, the system may optimize for clustering density or distances between feature centroids. This distinction is critical for Neuromorphic Engineering, as biological plasticity rules (like STDP) are unsupervised, functioning by detecting statistical correlations in the input stream to build internal representations without external labels.

4.2 • DEEP LEARNING FRAMEWORK

Modern Deep Learning aggregates simple units into high-dimensional layers. A deep network with L layers is expressed as a composite function mapping input $\mathbf{x}$ to output $\mathbf{y}$ through nested operations:

$$\mathbf{y} = f_L(\dots f_2(f_1(\mathbf{x})))$$

During the Forward Pass, each layer performs an Affine Transformation (a linear rotation and scaling of data via weight matrix $\mathbf{W}$ and bias $\mathbf{b}$) followed by a Non-Linear Activation (σ):

$$\begin{aligned} \mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= \sigma(\mathbf{z}^{(l)}) \end{aligned}$$

The non-linearity prevents the deep stack from collapsing into a single linear equation. Modern networks rely on the Rectified Linear Unit (ReLU), $f(x) = \max(0, x)$. Its derivative (0 or 1) preserves the magnitude of the gradient, allowing error signals to travel through deep structures without vanishing.

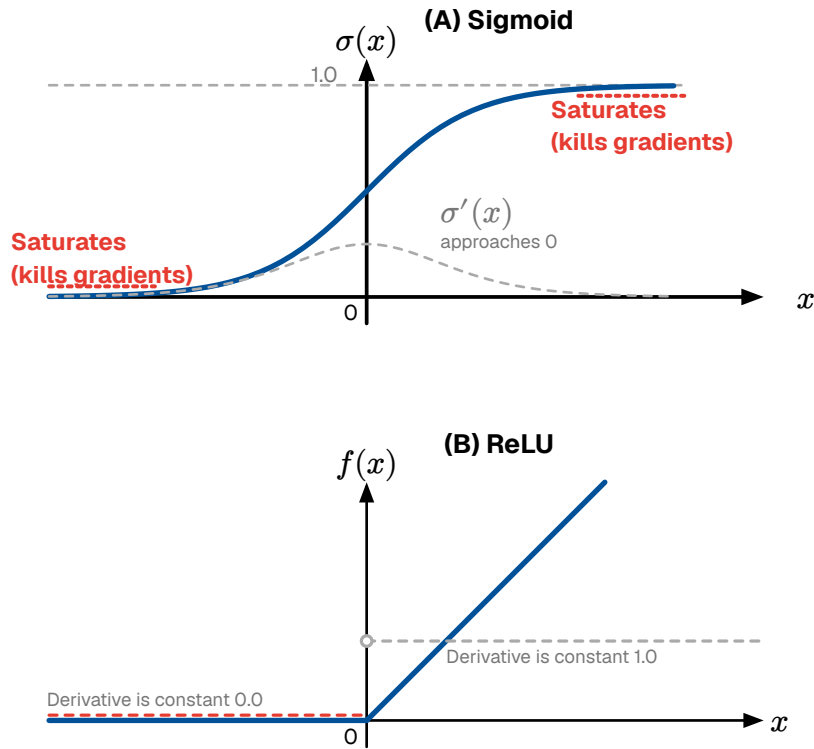


Figure 18: Activation Functions. The Sigmoid (left) saturates gradients. The ReLU (right) preserves gradient magnitude for positive inputs.

COMPUTATIONAL BOTTLENECKS: DENSE MATRICES

During the Backward Pass, Backpropagation recursively applies the Chain Rule via Automatic Differentiation to attribute the total error $J(\theta)$ to specific weights.

To achieve high throughput, these operations are vectorized. The affine transformation for an entire layer is executed as a Dense Matrix Multiplication (GEMM). This mathematical reality is the defining characteristic of modern AI hardware. A deep network is effectively a sequence of massive matrix multiplications. While highly parallelizable on GPUs, it creates a severe memory bandwidth bottleneck. The entire weight matrix $\mathbf{W}$ must be loaded into processor registers for every inference step, heavily contrasting with the sparse, localized updates of event-driven neuromorphic systems.

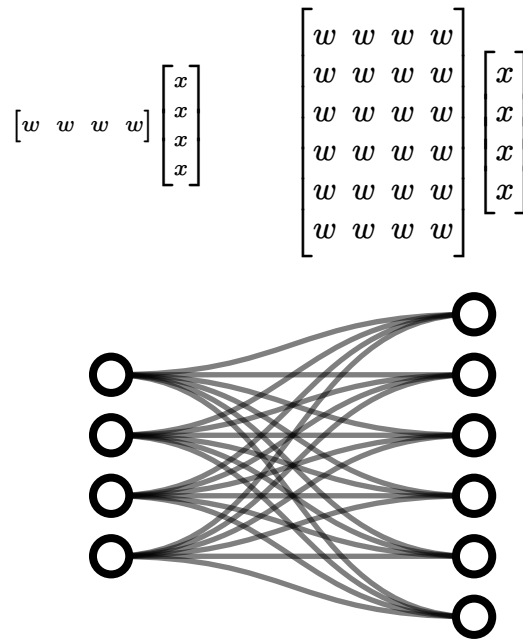


Figure 19: Deep Learning as Matrix Multiplication. Forward and backward passes rely on dense matrix products, necessitating high-bandwidth memory access.

CONVOLUTIONAL NEURAL NETWORKS (CNNs)

For visual tasks, standard Multi-Layer Perceptrons scale poorly; connecting every pixel to every neuron ignores the spatial structure of the data and creates an intractable number of weights. To solve this, DL utilizes CNNs.

CNNs apply small, learnable weight matrices known as “kernels” or “filters” that slide (convolve) across the input image. This architecture introduces two critical inductive biases:

1. **Local Connectivity:** Neurons only process a small, local receptive field, analogous to the biological visual cortex.
2. **Weight Sharing:** The exact same kernel is applied across the entire image, drastically reducing the number of tunable parameters and establishing translation invariance (a feature learned in one corner of an image can be recognized anywhere else).

While CNNs are the standard baseline for spatial processing, they remain fundamentally synchronous and frame-based, evaluating the entire image structure in dense mathematical passes regardless of local activity.

4.3 • WHY IS DEEP LEARNING INEFFICIENT?

While the matrix-centric formulation of Deep Learning enables high-throughput parallelization on GPUs, it fundamentally conflicts with the physical constraints of modern computing hardware. As models scale to billions of parameters, the primary bottleneck shifts from algorithmic capability to physical realizability. This inefficiency manifests in four distinct engineering dimensions:

THE VON NEUMANN BOTTLENECK & DATA MOVEMENT

The most significant physical limitation is the Von Neumann Architecture, which physically separates the Processing Unit from the Memory Unit. To perform a single inference step, the processor must fetch the entire weight matrix from off-chip DRAM to on-chip registers, perform the calculation, and write the results back.

According to Horowitz and Dally [1], fetching a 32-bit value from off-chip DRAM consumes approximately 640 pJ, whereas performing a floating-point addition on that value consumes only 0.1 pJ. The system expends 99.9% of its energy transporting data, and only 0.1% actually computing.

The Von Neumann Architecture & Bottleneck

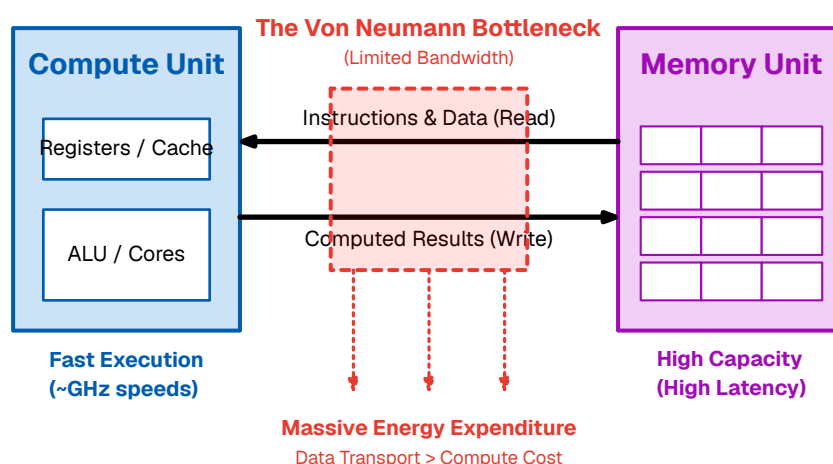


Figure 20: The Von Neumann Bottleneck. The separation of memory and compute forces massive energy expenditure on data transport.

DENSE PROCESSING OF SPARSE DATA

Standard Deep Learning implementations rely on Dense Matrix Multiplication (GEMM). This approach is algorithmically rigid: it executes the same number of operations regardless of the data content.

Real-world sensory data is often highly sparse, and the ReLU activation function naturally produces activation maps where the majority of values are zero. However, a standard GPU is “blind” to this sparsity. It will dutifully fetch a zero from memory and multiply it by a weight ($0 \times w = 0$), consuming energy and clock cycles to produce a null result. Deep Learning’s inability to exploit this silence represents a massive structural inefficiency.

THE HIGH COST OF SYNCHRONY

Deep Learning hardware is typically Synchronous, operating in lockstep with a global clock. Driving a high-frequency clock signal across an entire silicon die forces billions of transistors to charge and discharge continuously, regardless of whether

the chip is doing useful work. In high-performance processors, this clock distribution network alone can consume 30% to 40% of the total power budget. Furthermore, global synchronization enforces a “worst-case” latency: faster computations must sit idle and wait for the slowest operations to finish before the next clock cycle begins.

BACKPROPAGATION AND GLOBAL DEPENDENCIES

Finally, Backpropagation imposes severe constraints on memory and latency because it is non-local in both time and space. To update a specific weight, the system must wait for the Forward Pass to finish, calculate the global error, and wait for the backward pass to propagate the gradient.

This creates a “Locking Problem.” The activations of every intermediate layer must be stored in high-speed memory (VRAM) for the duration of the entire pass, preventing that memory from being reused. Additionally, a local synapse cannot adapt to local changes instantly; it is shackled to the global error loop.

4.4 • PRINCIPLES OF NEUROMORPHIC ENGINEERING

As established in the *History & Developments* chapter, Neuromorphic Engineering is the translation of biological dynamics into silicon hardware. It replaces the rigid, clock-driven logic of standard computing with the adaptive, event-driven dynamics of neural tissue. This approach rests on three architectural pillars that directly address the bottlenecks of Deep Learning:

- **Co-location of Memory and Compute (The Synaptic Principle):** Neuromorphic architectures eliminate the Von Neumann bottleneck by distributing memory across the silicon die. Each artificial neuron stores its own state and synaptic weights locally, processing data **in situ** to eliminate the energy cost of shuttling data.
- **Event-Driven Asynchrony (The Action Potential Principle):** Neuromorphic systems abandon the global clock. They operate asynchronously, driven strictly by the arrival of data. If a part of the network is not processing information, it consumes negligible power, ensuring energy scales linearly with task complexity rather than network size.
- **Sparse Communication (The Spike Principle):** Neuromorphic systems utilize binary Spikes for communication. Information is encoded in the precise timing of events rather than complex magnitudes, drastically reducing the bandwidth required to transmit information between neurons.

4.5 • TRAINING SPIKING NETWORKS: THE NON-DIFFERENTIABILITY PROBLEM

While the physical architecture of neuromorphic systems is highly efficient, training these networks presents a fundamental mathematical challenge. Standard deep learning relies on gradient descent, but backpropagation cannot be directly applied to native SNNs.

In a spiking network, the neuron's activation function is a discontinuous step function (the Dirac delta event threshold). The derivative of this function is zero everywhere except at the exact moment of the spike, where it is undefined. Consequently, gradients calculated using the chain rule immediately drop to zero—known as the “Dead Neuron” problem—preventing error signals from flowing backward through the network to update the weights.

To circumvent this non-differentiability and optimize network parameters, the field of neuromorphic engineering generally employs two distinct paradigms:

1. ANN-TO-SNN CONVERSION (DIRECT WEIGHT TRANSFER)

A pragmatic engineering approach to bypass the dead neuron problem is offline training. In this paradigm, a standard, continuous ANN (such as a network utilizing ReLU activations) is trained conventionally using backpropagation. Once convergence is achieved, the learned weights are directly mapped onto a structurally identical Spiking Neural Network.

The underlying premise is that the continuous activation values of the ANN can be approximated by the discrete firing rates of the SNN over a set time window. While this method allows the spiking system to inherit the high accuracy of gradient descent, direct weight transfer requires careful scaling and normalization. If the weights are copied without adjustment, the resulting SNN may suffer from catastrophic saturation (firing constantly) or severe signal degradation (failing to reach the spiking threshold).

2. NATIVE LOCAL LEARNING (STDP)

To fully exploit the energy efficiency and event-driven dynamics of neuromorphic hardware, training must ideally occur natively on the spiking substrate. This requires abandoning global backpropagation in favor of biologically plausible, mathematically local learning rules.

As established in Section 3.6, Spike-Timing-Dependent Plasticity (STDP) adjusts synaptic weights based strictly on the temporal correlation of local pre- and post-synaptic spikes. Because STDP relies exclusively on local physical events rather than global error gradients, it does not require a differentiable loss function. This allows

the network to completely bypass the dead neuron problem, enabling unsupervised feature extraction and real-time adaptation directly on the spiking architecture.

3. SURROGATE GRADIENT DESCENT

For completeness, it must be noted that the current dominant paradigm in SNN research utilizes Surrogate Gradients. In this approach, the network operates using the discontinuous spike step-function during the forward pass, but temporarily replaces the undefined derivative with a smooth, continuous approximation (a “surrogate”) during the backward pass. While this thesis focuses on evaluating direct weight transfer and native unsupervised STDP, surrogate methods represent a highly effective hybrid approach, allowing backpropagation-like algorithms to estimate gradients across discrete spiking layers.

4.6 • NEUROMORPHIC HARDWARE TECHNIQUES

Central to realizing these computational efficiencies in physical hardware is Address Event Representation (AER), a communication protocol that mirrors the sparse nature of biological spikes. Instead of continuous data streaming, the hardware only transmits the “address” of a firing neuron across a shared digital bus, allowing a single physical wire to represent thousands of virtual axonal projections.

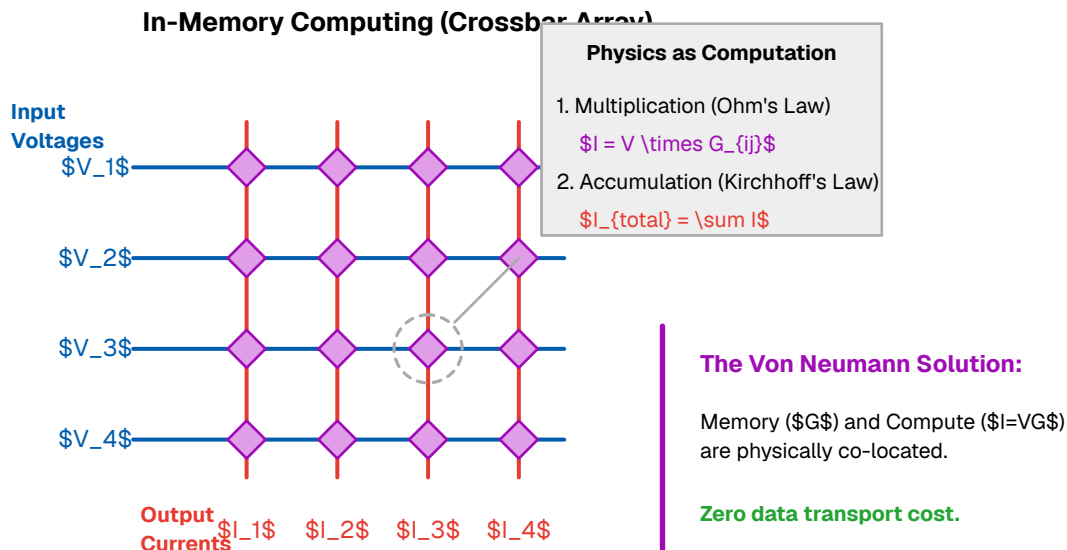


Figure 21: In-Memory Computing via a Crossbar Array. Unlike von Neumann architectures, memory and computation are physically co-located. Input voltages (V) are applied to the wordlines. Memory elements at the junctions hold programmable conductances (G). Multiplication is natively performed at each junction by Ohm’s Law ($I = V \times G$), and resulting currents are summed along the bitlines via Kirchhoff’s Current Law. This allows dense matrix-vector multiplications to occur in a single analog time step with zero data transport cost.

The crossbar array provides a direct structural surrogate for the neural neuropil. Because the architecture handles multiplication and summation natively through

physical laws, it is uniquely suited to implement biological “macro-motifs.” By routing bitline currents through local feedback loops, the hardware can instantiate complex dynamics such as Lateral Inhibition and Winner-Take-All circuits without the overhead of high-level software instructions.

This synergy between physical topology and functional motifs allows the hardware to inherit the computational efficiency of the neocortex, effectively making the architecture itself the algorithm.

Hierarchical Architecture & Address Event Representation (AER)

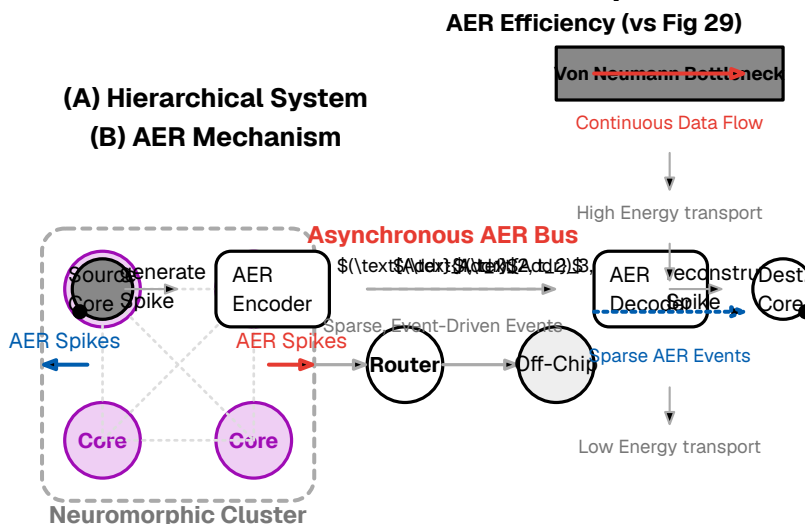


Figure 22: Architectural Comparison. (Left) The Von Neumann architecture separates memory and compute, creating a bottleneck. (Right) The Neuromorphic architecture co-locates them, mimicking the distributed topology of biological neural networks.

4.7 • STATE OF THE ART: SENSORS AND SIMULATION

The principles of neuromorphic computing have matured from theoretical biophysics into a vibrant field of applied engineering. While the fundamental architecture relies on the co-location of memory and compute, interacting with the real world requires a paradigm shift in how data is acquired and how these networks are prototyped.

EVENT-BASED SENSING: THE DYNAMIC VISION SENSOR (DVS)

Traditional computer vision relies on Active Pixel Sensors (APS) that capture the state of the entire environment at fixed temporal intervals (frames). While effective for static analysis, this generates highly redundant data streams that impose significant latency and power penalties.

Neuromorphic sensors abandon the global shutter in favor of asynchronous, event-driven acquisition. The most mature realization of this paradigm is the Dynamic Vision Sensor (DVS), or event camera. Each pixel in a DVS operates independently,

monitoring the log-intensity of incident light. An event e is generated at time t only when the brightness change exceeds a preset threshold θ :

$$\Delta \ln(I) = \ln(I(t)) - \ln(I(t - \Delta t)) \geq \pm \theta$$

This mechanism yields a sparse stream of events characterized by the tuple (x, y, t, p) , representing the pixel coordinates, the microsecond-resolution timestamp, and the polarity of the change (ON or OFF). Because data is only generated when the scene changes, DVS cameras achieve kilohertz-equivalent frame rates, massive dynamic range (> 120 dB), and extreme energy efficiency. Consequently, event streams serve as the natural input data for Spiking Neural Networks.

HARDWARE VS. SOFTWARE SIMULATION

Deploying SNNs to process these event streams is currently achieved through two distinct avenues: dedicated physical hardware and software simulation.

At the hardware level, systems like IBM’s TrueNorth and Intel’s Loihi physically instantiate the neuromorphic principles discussed previously. TrueNorth functions as a highly efficient digital inference engine, utilizing over a million neurons to process data at milliwatt power budgets. Loihi expands upon this by including programmable learning engines within its cores, allowing for the real-time, on-chip execution of local plasticity rules like STDP.

However, before algorithms can be deployed to specialized silicon, they must be rigorously validated. This is achieved through SNN software frameworks that simulate event-driven dynamics on standard Von Neumann hardware (CPUs/GPUs). Frameworks such as Brian2 provide deep biophysical realism for computational neuroscience, while libraries like Nengo, snnTorch, and BindsNET bridge the gap to machine learning, allowing spiking dynamics to be integrated with standard deep learning data pipelines.

Having established the theoretical limitations of traditional deep learning and the physical principles of neuromorphic engineering, the following chapter details the specific methodologies, network architectures, and software frameworks utilized in this thesis to evaluate ANN-to-SNN weight conversion and native STDP learning on visual event data.

5 • METHOD

This chapter details the specific implementations of the neuromorphic architectures proposed to address the limitations of standard deep learning. Aligning with the biological constraints of sparsity, asynchrony, and locality established in previous chapters, we outline the construction and evaluation of a Spiking Neural Network (SNN).

To empirically validate the theoretical advantages of neuromorphic algorithms, we evaluate the system on a benchmark image classification task. The experiment is bifurcated into two distinct phases to compare offline engineering with native biological learning:

1. **Inference via Weight Transfer:** Evaluating the zero-shot performance of an SNN initialized with weights directly mapped from a classically trained Artificial Neural Network (ANN).
2. **Native Unsupervised Learning:** Training an SNN from scratch utilizing local Spike-Timing-Dependent Plasticity (STDP).

5.1 • DATASET & PRE-PROCESSING

To benchmark these algorithms, we require a dataset that necessitates the extraction of complex spatial features but remains computationally tractable for rapid experimental iteration. We utilize the MNIST database of handwritten digits [1].

The dataset consists of a training set of 60,000 examples and a test set of 10,000 examples of digits (0-9). Each instance is a 28x28 pixel grayscale image. While standard deep learning models routinely score over 90% accuracy on this task, making it largely a solved problem in classical AI, its well-understood feature space makes it an ideal, isolated baseline for verifying novel neuromorphic architectures.

Because SNNs operate on discrete events rather than continuous values, the raw pixel intensities (ranging from 0 to 255) must be pre-processed and translated into spike trains before they can be ingested by the network.

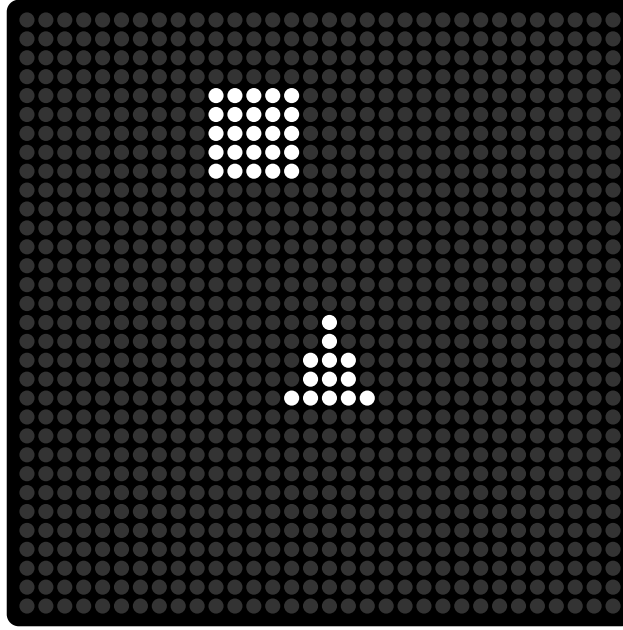


Figure 23: Sample of the MNIST dataset. Pixel intensities must be translated into temporal spike events.

5.2 • NETWORK ARCHITECTURE

To facilitate a direct, one-to-one mapping of synaptic weights from the offline Artificial Neural Network (ANN) to the native Spiking Neural Network (SNN), both models must share an identical macroscopic topology. Therefore, this implementation utilizes a Fully Connected Network (Fully Connected Network (FCN)), also known as a Multi-Layer Perceptron (MLP), rather than a Convolutional Neural Network (CNN).

While CNNs are the standard baseline for vision tasks due to their spatial inductive biases, transferring convolutional kernels into a spiking substrate introduces significant mapping complexities—specifically the need to physically unroll and duplicate shared weights across the spiking array. An FCN provides a straightforward, mathematically transparent architecture for cleanly evaluating direct weight transfer and STDP without confounding architectural variables.

The network is structured as a shallow hierarchy to capture the primitive geometric features of the dataset:

- **Input Layer:** The 28×28 pixel grayscale images are flattened into a 1D vector, requiring 784 input neurons.
- **Hidden Layer(s):** Fully connected intermediate layers designed to extract and recombine features (e.g., combining simple edges into loops and lines).
- **Output Layer:** 10 neurons, corresponding directly to the categorical digit classes (0 through 9).

Convolutional Neural Network (CNN) Architecture

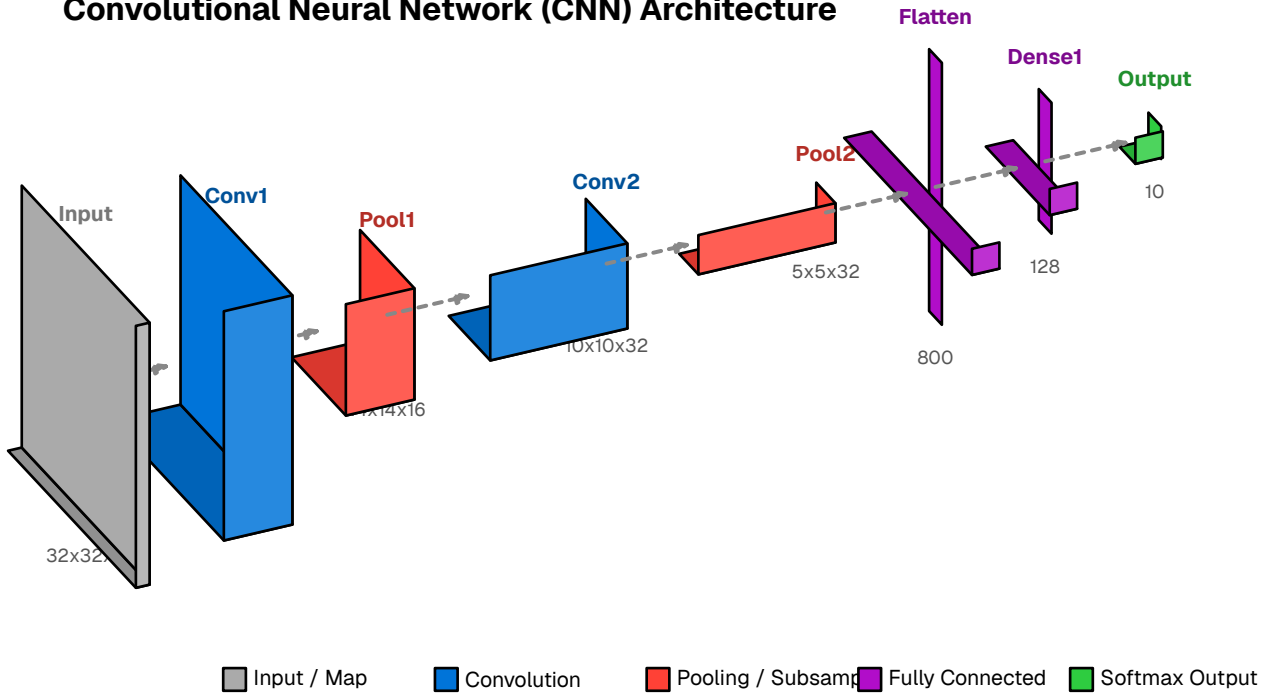


Figure 24: Network architecture. The 28×28 images are flattened and passed through a Fully Connected Network. Both the ANN and SNN share this identical macroscopic topology.

Despite the macroscopic symmetry required for weight sharing, the microscopic dynamics of the two networks differ fundamentally. The offline ANN utilizes standard continuous activation functions (such as ReLU) to calculate smooth gradients during backpropagation.

In contrast, the SNN replaces these continuous functions with Integrate-and-Fire neurons governed by a strict voltage threshold. This simulates the biological “all-or-nothing” action potential, acting as a hard step function. Furthermore, the spiking architecture utilizes lateral inhibition at the output layer. This engenders a Winner-Takes-All (WTA) dynamic; as the network integrates evidence over time, the first output neuron to fire strongly inhibits its competitors, forcing a definitive categorical decision and actively suppressing noise.

5.3 • INFORMATION REPRESENTATION

The choice of neural code lays the foundation for information flow and dictates the efficiency of the entire system. While Rate Coding (encoding pixel intensity as spike frequency) is straightforward and simple to implement with standard Integrate-and-Fire neurons, it is highly inefficient. Rate codes require integration over extended time windows to calculate an average, introducing latency and saturating the network bus with redundant spikes.

To maximize energy efficiency and processing speed, this implementation utilizes a Time-to-First-Spike (TTFS) temporal encoding. In this regime, a single spike carries the information payload. A high-intensity (bright) pixel triggers an early spike,

while a low-intensity (dark) pixel triggers a late spike. This compresses the spatial information into a highly sparse, priority-driven queue; downstream neurons begin processing as soon as the most salient features arrive, without waiting for an entire frame to integrate.

As noted in Section 3.4, temporal codes suffer from Phase Ambiguity—downstream neurons need a reference “clock” to decode latency. To resolve this without relying on a rigid, global system clock, we simulate the biological concept of a **saccade** (the rapid movement of the eye to fixate on a target). The initial presentation of the image acts as a synchronized global event (t_0). All subsequent input spikes are evaluated relative to this saccade onset, providing a natural, biologically plausible temporal reference frame.

The choice of a coding scheme puts key constraints on the design of any processing system as it lays the foundation for the flow of information. In Section 3.4 we discussed several candidate neural codes that are both biologically plausible and have been used in previous neuromorphic systems. The neural code plays a large role in determining the efficiency of the system. Many neuromorphic systems use rate code as it is easy to translate values; it is straightforward. Any float value can be encoded as a rate code and integrate-and-fire neurons work well with this encoding. An IF neuron using rate code can reduce the multiply and add operations with just add operations. Each spike arriving at a synapse adds its weight to the total. It is easy to see that for a rate code a higher rate input means more of that input contributing to the total sum. Using a rate code we can convert multiply and accumulate operations to just a series of accumulate operations by discretizing the input values. The main reason for why this thesis will not use a rate code is that it is relatively inefficient and slow compared to a temporal code. In a temporal code only one spike is required to establish its value in relation to other synapses, making a single event much more dense in information. This is especially important for systems that do not have the connection density of a biological brain and must use shared buses that can be congested if too many spikes are present on the bus. Secondly, to determine the value of a rate code you have to take an average of multiple spikes, imposing a delay. Using a TTFS encoding requires more sophisticated neuron models than a simple integrate-and-fire; it needs more bookkeeping to keep track of the relative arrival of incoming spikes and as mentioned in Section 3.4 and as we will explain by using a temporal encoding we need to handle the phase ambiguity problem.

to fire. The problem with this is that if we have a short one spike first that does not make the neuron fire but just before the timer runs out we get a large burst that would make the neuron fire but since the first isolated one triggered the counter we do not get a output spike. However I think this is fixed with the “clock” being a saccade to sync and fix the phase ambiguity.

A neural code should have the following characteristics:

- Fast
- Efficient in terms of neurons used
- Able to encode a wide range of stimuli
- Robust to noise

Visual tasks which is what the application this thesis focuses on

1. a population code is used in conjunction with a temporal code
2. Population code is used alone (eg individual order within a population does not matter)

That being said and as discussed in Section 3.4 combining neural codes and using the right one for the job is also an important aspect to consider, there is not a clear consensus on which code is better it is a matter of the task at hand. As we also have talked about, Combining different codes together can make for a powerful representation of information. This thesis focuses on visual recognition and a TTFS combined with population code is the representation of choice. The population code will represent a distinct pattern in an image. This can be set up to detect lines and primitive shapes, deeper layers will use populations of primitive shapes to detect more complex features like circles and boxes and so on.

5.3.1 • ENCODING

To convert a float value (like a pixel value from an image to a TTFS encoding we first need to find the range of the pixels from minimum to maximum value. Then we need to decide on what feature of interest we wish to represent, a natural choice is pixel luminance but others such as contrast or hue or other features using other color spaces could be used. For our purposes we will not be using contrast but since our image is engineered for this task we can skip the contrast extraction that needs to be in place for real world images and produce images that has clear distinct lines present in the raw pixel data (since the dimensions are small a line in the image will typically only be a few pixels wide so there would be no difference if we took the contrast of pixels or the luminance directly).. If the image is black and white it is straight forward to extract the luminance as most often the pixel luminance is stored directly. Once we have found the range say minimum is 0 and max is 255 we can construct an event queue where the pixel intensity determines its place in the queue and its associated delay. The conversion mapping can be linear where each pixel gets its luminance multiplied by a scalar other useful mappings can be logarithmic or exponential.

```

    procedure intensity_to_delay_encoding(image, T_max=100, T_min=0):
1   |   normalized_image = normalize(image)
2   |   spike_times = T_max - (T_max - T_min) * normalized_image
3   |   return spike_times

```

Algorithm 1: Linear intensity to delay encoding

```

    procedure intensity_to_delay_encoding(image, T_max=100, T_min=0):
1   |   normalized_image = normalize(image)
2   |   spike_times = T_max - (T_max - T_min) * normalized_image
3   |   return spike_times

```

Algorithm 2: Logarithmic intensity to delay encoding

In deeper layers the meaning of the information may be obscured as is common with deep neural networks, however temporal encoding should if the neuron model is designed with this scheme in mind should produce shorter delays for greater values if a deeper layer neuron reacts to a box and its neighbour in the same layer reacts to a circle and the image is a rounded box (but more box than circle) then the neuron responding to a box should emit a spike faster than the neuron responding to a circle.

5.3.2 • DECODING

Decoding the neural code requires an adequate neuron model that uses the neural code to run useful computations. As discussed in Section 3 a neuron should have the following characteristics:

- Accumulate spikes in some form (integrate)
- Fire when some threshold has been reached
- Leak the potential over time

We have seen that glif neuron is bio plausible and such a neuron is simple and can be realized easily on hardware the way. Using a temporal code the neuron has to differentiate between inputs in the time domain otherwise it cannot decode the information and cannot do useful work. Earlier inputs are encoded with larger values and should have a greater portion of summation going on inside the neuron. The neurons need to be designed to work with the chosen encoding either at individual neuron level or population level

-If order matters (temporal code) the neuron must handle it Evidence for bio-plausibility can be found from eq 1 where the $R(u)$ is dependent on the membrane potential

make the input exponentially weaker in proportion to the threshold. If the threshold is zero then the incoming spike does not get suppressed if the threshold is very close to max then the incoming spike gets more suppressed.

Leaky Membrane Potential Dynamics

$$V_m(t) = V_{rest} + \sum w_i \exp\left(-\frac{t - t_i}{\tau}\right)$$

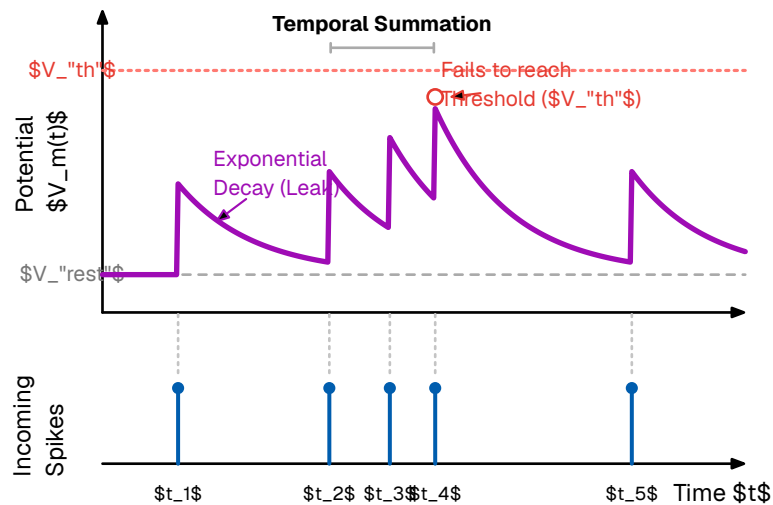


Figure 25: Neuron model where new incoming spikes have an exponential decaying influence on the potential

A counter starts when the first spike arrives

Needs a global reset or local like we talked about in the biological section about phase ambiguity

```

integrate_and_fire(excitatory, inhibitory, threshold) → integer:
1  excitatory_spikes = []
2  if inhibitory is None:
3      inhibitory_spikes = []
4  else:
5      inhibitory_spikes = []
6  all_spikes = excitatory_spikes + inhibitory_spikes
7  if not all_spikes:
8      return None
9  all_spikes.sort(key=lambda x: x[0])
10 integrated_potential = 0.0
11 firing_time = None
12 for time, spike_type in all_spikes:
13     integrated_potential += spike_type
14     integrated_potential = max(0, integrated_potential)
15     if integrated_potential ≥ threshold:
16         firing_time = time
17         break
18 return firing_time

```

Algorithm 3: Logarithmic intensity to delay encoding

In a time to first spike scheme if we care about the order (the relative values since information is stored in time and order) we have to use weights and a neuron model

that distinguish between inputs arriving earlier than others. I present a scheme where the first neuron that arrives starts a linear count where the slope of the counter is the weight additional inputs will increase or decrease the slope according to their weight. We can see that neurons arriving earlier will get more time to increase the counter and thus will carry a higher value. If the counter reaches a threshold the neuron will fire. The astute will notice that in this scheme the neuron will fire even for the smallest stimulus since the counter will count up a non zero value and eventually reach the threshold, to mitigate this we can simply say that if the counter is too slow the neuron will not fire we will see later that this scheme satisfies the criteria above.

The problem with this decoding is for strong stimuli we would ideally make the neuron respond immediately and fire, but it has to wait until the counter has reached the threshold to fix this we can also add the weight of the input directly to the potential while also starting a counter. Now if early strong inputs arrive they will fill up the potential and make the neuron fire almost immediately. Small inputs will take some time

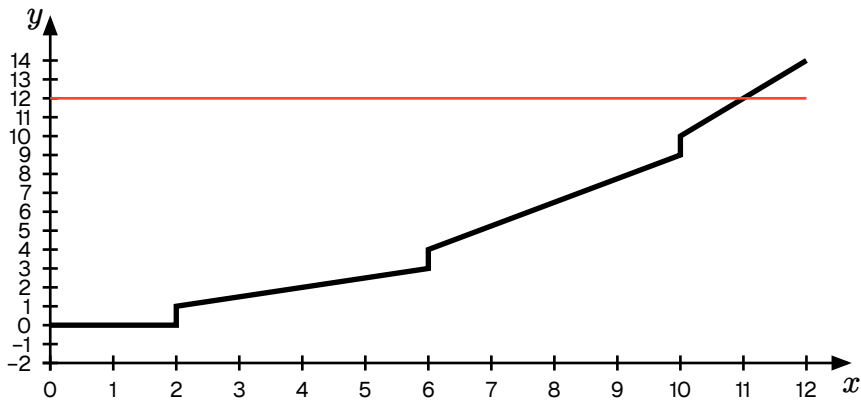


Figure 26: Proposed simplified layout of a SNN. The neurons are connected with hierarchical busses that allow for the network to be configured as a *small world network*

recall this equation. TTFS model should mathematically behave the same

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Leaky integrate and fire models seem the best bet, however complex dynamics like exponential decay and analog weights and potentials seem excessive, we might do without. Binary weights 1 for excitatory and 0 for inhibitory. Stronger weights can be modeled with multiple parallel synapses

Another way which is also based on relative firing order of single spikes could be a passcode encoding. Such an encoding could work by having neurons only react to a sequence. It has an internal state machine of sorts and will only advance to the next state if it receives the correct input in the correct order. This encoding does only care about relative order not relative timings.

5.4 • SNN IMPLEMENTATION DETAILS

5.5 • TRAINING METHODOLOGIES

start with fully connected and then run strengthen connections that have “statistical significance” (not random / patterns) or is correlated to something (reward signal) at the end of an epoch prune connectionn making the network more effecient. This is similar to how the brain does it when sleeping

Do some math with references to the optimasation section

If the post neuron fires then we should strengthen the weights. Synapses gets grown at random this might be inneficient but it is highly parallelizable an idea could be to grow synapses with a kind of gravity where post synapses has a pulling effect if they are already strong

The encoding fit for images are a kind of population code where a pattern comes at the same time. The relative timing between patterns is used for strength earlier patterns are stronger. In a inhibitory network this allows for winner take all

learning with binary weights

learning with more steps of quantized or continous weights

Say we want to detect the pattern ABC and the pattern ABD. First of all if the order does not matter set all the weights equal. If the order does matter the weights determine the order. Now if a neuron learns pattern ABC so well that it learns to fire on only AB then it can fire faster. However if a second neuron wants to learn ABD then inhibition from the AB neuron prohibits it. A solution can be that if a neuron originally learned ABC but now fires on AB but stil has a strong weight on C it should remember this and if it fires on AB but then C does not arrive it should be like “oh, C did not show maybe I am wrong to fire early” eg. Decrease weights for A and B It predicts!

A second way is to have a hierarchy with bypass. So one layer detects only AB then the next layer has bypass of the first layer and the second combining AB and C or D

A second problem is how to decode order. When do we start the decreasing timer, how fast, should it be in time or in amount of spikes, what to do with phase? The phase should correct itself. The weights need to be as presise as the timing of the spikes? Or we could make the neuron sensitivity proportional to its inverse potential and add leaking

5.5.1 • WEIGHT TRANSFER

```
1 start with a collection of neurons with arbitrary connections
2 if a pre-synaptic neuron fires then
3   | it has a chance to grow a synapse to a random post-synaptic neuron
4 if a post-synaptic neuron fires then
5   | strengthen all connections to pre-synaptic neurons that fired before
6   | remove connections to pre-synaptic neurons that did not fire or fired
   | after
   ⓘ a neuron can be both pre-synaptic and post-synaptic
```

Algorithm 4: Unsupervised local learning rule for individual neurons. Based on STDP

```
1 probability of growing a synapse is inversely
   proportional to the amount it already has
2 earlier firings should get a better chance to grow synapses,
   although this is regulated by inhibitory action
```

Algorithm 5: Growing rules for synapses

5.5.2 • STDP INSPIRED

5.6 • SIMULATION SETUP

We have discussed the benefits of co algorithm design and designing new speialed computer hardware that run neuromorphic algorithms directly on the substrate via gates or analaog elements and exotic new materials. However developing biologically inspired computers and algorithms on cheap and avaiable CPU and GPU hardware is a great way to quickly iterate and test

Altough many simulation and software packages exists as outlines in. The methods in this thesis has been tested using custom simulaton software for full control

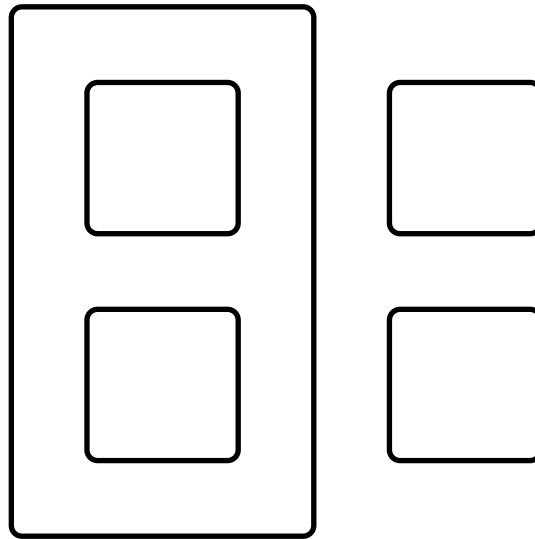


Figure 27: Simulator architechture block diagram

The simulator runs an event loop Spikes are pushed to a heap The simulator can run both on CPU and GPU, When running on CPU the spikes are pushed to a heap, when running on GPU spikes are pushed to a adress event bus the algorithms presented in this thesis are highly paralizable It is built from the ground up using the Vulkan API



Figure 28: In-memory

5.7 • EVALUATION METRICS

To test and verify the methods we need a way to measure the performance for classification tasks accuracy recall and . is often used. This works great for supervised learning. For unsupervised learning ...

Accuracy and recall measures effectiveness but as the goal of this thesis is to improve efficiency we need a way to measure the resource usage. There are direct ways to do this like measure power draw and data usage however we do not have the resources to set up such a test rig. Another way is the measure and theorize about number of operations as an indirect measure of resource usage. This has several accuracy issues but can give an estimate. The first issue is that this is not an apples to apples comparison

6 • RESULTS

6.1 • INFERENCE

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

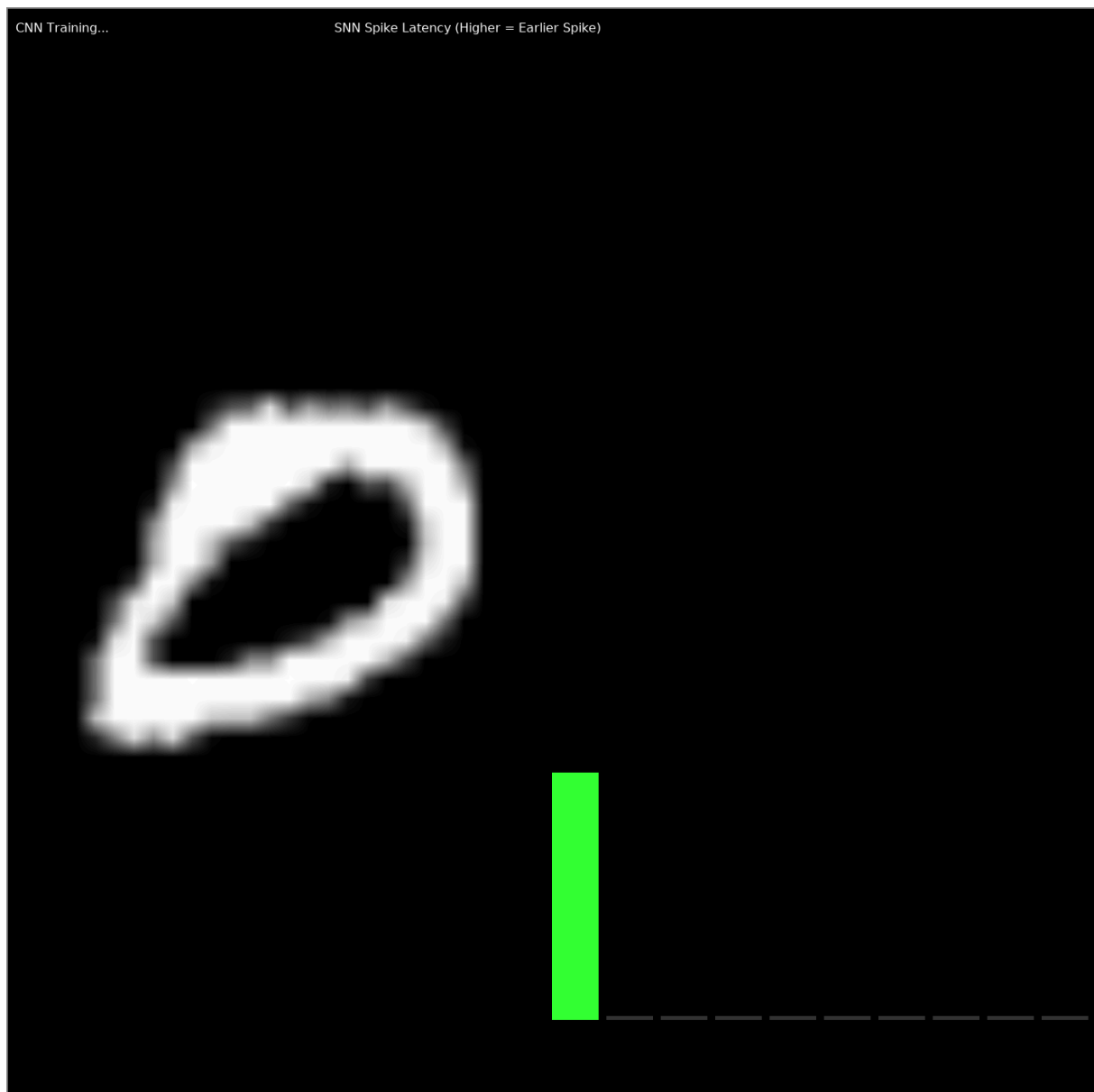


Figure 29: Neural network before learning

SNN Quantized Synapse Maps (i8 Precision)

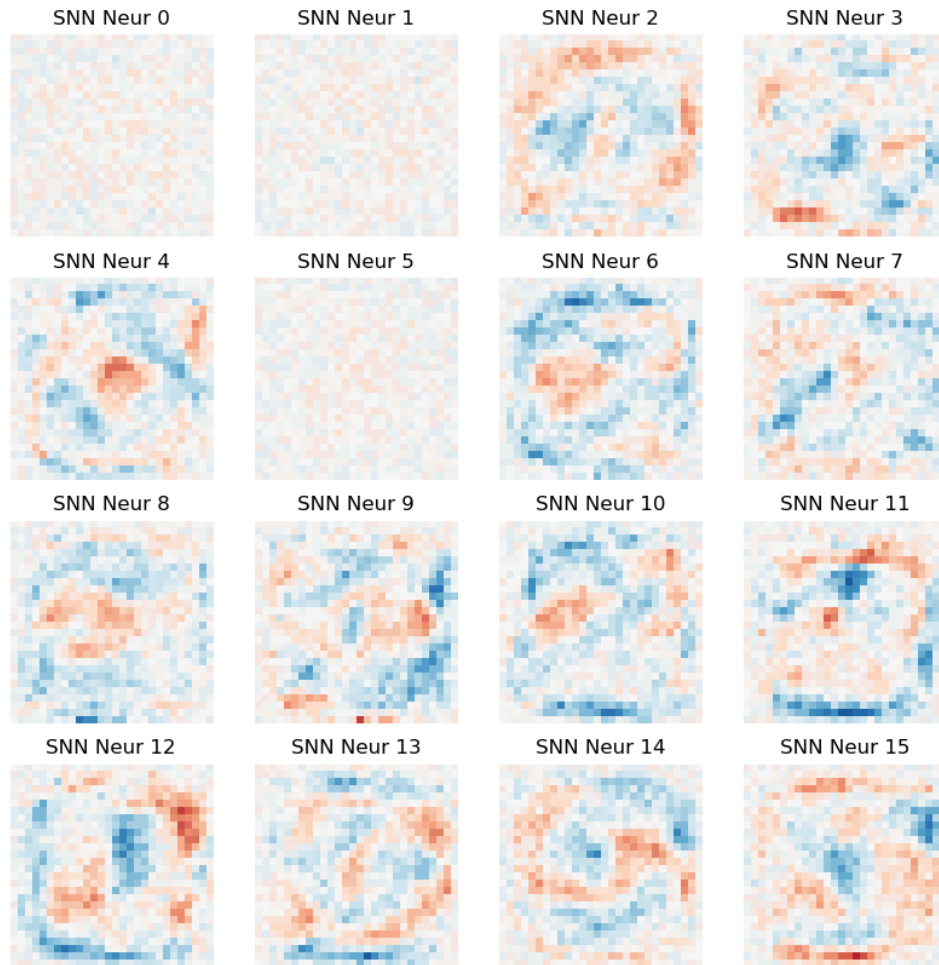


Figure 30: Neural network before learning

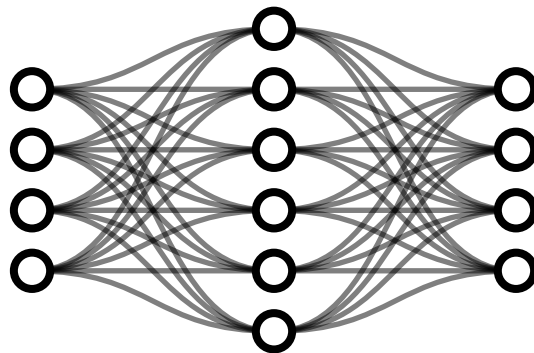


Figure 31: Neural network during learning

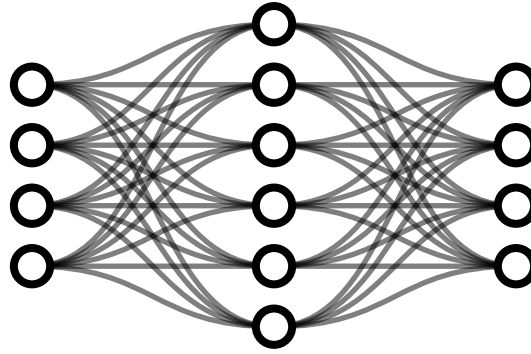


Figure 32: Neural network after learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamur animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamur animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.2 • TRAINING

0	0	1	1
1	1	1	1

Table 1: Number of operations

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamur animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum

id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in

quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7 • DISCUSSION

In this section we discuss the

In the engineered test environment we used pixel intensity directly and for this toy example it works quite well since the features we are interested in are easy to read in a low resolution simple environment. However for real world vision tasks pixel intensity is not the best features to feed into the algorithm, it is better to use local contrast since that is what determines object outlines and other interesting elements of an image. It is possible to use an SNN to compute local contrast from pixel intensity, in fact that was part of the development of the algorithm. For a TTFS approach it gets very difficult to do this efficiently since to get the normalized contrast (true contrast independent of luminance) you have to wait for all signals to arrive even the complete absence of light completely wasting the potential of TTFS. To fix this a dedicated chip that outputs TTFS contrast directly can be used. A normal difference of luminance gives you an absolute measure of contrast where differences of brighter pixels gives a larger contrast where differences between two darker pixels gives a lower contrast. Contrast should be relative (a ratio) so by converting luminance to log-space first and then taking difference will give a true contrast independent of luminance intensity

We see a trade-off between the ability to learn and speed of learning and forgetting. Synaptic plasticity must be tuned in order for the right learning environment to form representing contrast is difficult with a ttfs scheme, better to use special sensors and pass the encoded contrast in ttfs to a neuromorphic processor

representing position in a ttfs scheme proved difficult, precise timings and clever encodings are needed to map a delay to a coordinate. in biology section we briefly discussed population coding in the visual and motor cortex where neurons are orthogonal and encode a direction, however these schemes seem to work best with rate encoding as the intensity of a direction is more straight forward to encode, simply increase firing rate for that neuron, with ttfs we need a reference signal and have to wait for the slowest neuron. for numerical values rate encoding seems best for categorical and decision making ttfs is good.

Another way may be to have a hierarchy of “space cells” say each in a grid of 8x8 followed by another 8x8 that way we have 8x8 + 8x8 rather than 32x32 ofc we still have limited resolution at the borders

translating cnn to snn is not straight forward, cnn have negative weights snn does not, cnn can use pooling kernels etc snn does not use the same things. things like lateral inhibition is not directly translatable

7.0.1 • PERCEPTRON EQUIVALENCE

The perceptron equation can be obtained with a ttfs using the inverse of firing times

$$T = \sum \frac{w}{t}$$

Equation 9: Weigthed sum

In a time to first spike scheme of we care about the order (the relative values since information is stored in time and order) we have to use weights and a neuron model that distinguish between inputs arriving earlier than others. I present a scheme where the first neuron that arrives starts a linear count where the slope of the counter is the weight additional inputs will increase or decrease the slope according to their weight. We can see that neurons arriving earlier will get more time to increase the counter and thus will carry a higher value. If the counter reaches a threshold the neuron will fire. The astute will notice that in this scheme the neuron will fire even for the smallest stimulus since the counter will count up a non zero value and eventually reach the threshold, to mitigate this we can simply say that if the counter is too slow the neuron will not fire we will see later that this scheme satisfies the criteria above.

The problem with this decoding is for strong stimuli we would ideally make the neuron respond immediately and fire, but it has to wait until the counter has reached the threshold to fix this we can also add the weight of the input directly to the potential while also starting a counter. Now if early strong inputs arrive they will fill up the potential and make the neuron fire almost immediately. Small inputs wil take some time

recall this equation. TTFS model should mathematically behave the same

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Leaky integrate and fire models seem the best bet, however complex dynamics like exponential decay and analog weights and potentials seem excessive, we might do without. Binary weights 1 for excitatory and 0 for inhibitory. Stronger weights can be modeled with multiple parallel synapses

Another way which is also based on relative firing order of single spikes could be a passcode encoding. Such an encoding could work by having neurons only react to a sequence. It has an internal state machine of sorts and will only advance to the next state if recives the correct input in the correct order. This encoding does only care about relative order not relative timings.

easy with rate code

need global information to normalize

using ttf is difficult

using negative image - still problem with absolute contrast

can be done locally with speial sensors and using logarithmic intensity to delay

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et

collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7.1 • FUTURE WORK

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

8 • CONCLUSION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae

doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

BIBLIOGRAPHY

[1] ?, "Placeholder," 2025.